

Universidad Tecnológica de Panamá
Facultad de Ingeniería de Sistemas Computacionales
Departamento de Simulación y Computación de
Sistemas

Proyecto:
SIMAP Digital · Cuentas Claras

Tema:
Documento de Requerimientos y Arquitectura de Software

Asignatura:
Ingeniería de software II

Facilitador:
Ing. Jorge A. Marín

Grupo:
Josie Florez
Celeste Pérez
Adrián Pérez

Fecha de Entrega:
14 de junio de 2026

Tabla de contenido

1. Propósito
2. Alcance Del Producto / Software
3. Referencias
4. Funcionalidades Del Producto
5. Visión General
6. Clases Y Características De Usuarios
7. Entorno Operativo
8. Requerimientos Funcionales
9. Reglas De Negocio
10. Requerimientos De Interfaces Externas
11. Requerimientos No Funcionales
12. Otros Requerimientos
13. Posicionamiento
14. Descripción De Stakeholders Y Usuarios
15. Descripción Global Del Producto
16. Características Secundarias
17. Restricciones
18. Precedencia Y Prioridad
19. Otros Requisitos Del Producto
20. Modelo Casos De Uso
21. Modelo De Arquitectura De Análisis
22. Modelo De Base De Datos
23. Correspondencia De Necesidades De Actores Y Casos De Uso
24. Actores Y Sus Responsabilidades
25. Modelo De Análisis
26. Análisis De Relaciones Entre Modulos
27. Modelo Ossim
28. Documentación De La Arquitectura
29. Vistas Glosario Historial De Versiones Información Del Proyecto
- Aprobaciones

1. Propósito

El propósito de este documento es definir de forma meticulosa y exhaustiva la especificación de requerimientos y la arquitectura de software del sistema SIMAP Digital · Cuentas Claras. Este documento describe todos los módulos operativos, financieros y de inteligencia artificial de la Plataforma Web Progresiva (PWA) Offline-First diseñada para la modernización de las Juntas Administradoras de Acueductos Rurales (JAAR) en la República de Panamá.

2. Alcance del producto / Software

SIMAP Digital es un sistema integral tecnológico que sustituye el registro manual tradicional en cuadernos por un libro mayor digital.

- Propósito: Optimizar y transparentar la administración contable (cobro de tarifas de agua) y la gestión de jornales (trabajo comunitario) en zonas sin conexión a internet.
- Beneficios: Garantiza el pleno cumplimiento del Decreto Ejecutivo N° 1839 del Ministerio de Salud (MINSA). Mitiga la desconfianza local mediante la rendición sistemática de cuentas. Reduce la morosidad utilizando gamificación (puntos por pago puntual) y herramientas analíticas (Ruta IA).
- Objetivos: Digitalizar la contabilidad rural, asegurar operatividad en áreas sin conectividad (Offline-First), sistematizar el control de jornales, y automatizar la rendición de cuentas (PDF/Excel) ante el MINSA.

3. Referencias

- Decreto Ejecutivo N° 1839 del 5 de diciembre de 2014 (MINSA, Panamá).
- Documento: *JAAR_Digital_Propuesta_Final.md*.
- Documento: *casos_de_uso_v2.md*.
- Documento: *diagrama_er.md*.
- Documentación de Dexie.js (IndexedDB wrapper) y Supabase (BaaS).

4. Funcionalidades del producto

1. Registro y Autenticación Multi-Rol.
2. Cobranza Offline-First (Pagos Estándar, Parciales, Adelantados y Diarios).
3. Sincronización Automática (IndexedDB a Supabase).
4. Gestión y Registro de Jornales Comunitarios y Multas.
5. Motor de Gamificación (Puntos por pagos y asistencia, canjeables por descuentos).
6. Split de Comisiones automático para cobradores.
7. Inteligencia Artificial (Ruta IA, predicción de morosidad y cálculo de riesgo).
8. Reportes Oficiales MINSA (PDF y XLSX).

5. Visión General

El resto del documento presenta las reglas de negocio, características detalladas de los usuarios, la lista estructurada de requerimientos funcionales y no funcionales, así como el modelado de datos (ER), diagramas de casos de uso y la arquitectura del sistema basada en vistas lógicas y físicas (Modelo 4+1).

6. Clases y características de usuarios

1. Administrador Global (admin): Control total de finanzas, tarifas, auditoría, revisión del tablero de métricas IA y gestión (aprobación/rechazo) de usuarios.
2. Cobrador en Campo (cobrador): Registra pagos y jornales de forma offline, recibe comisiones automáticas, utiliza “Ruta Inteligente” de la IA para priorizar cobros.
3. Auditor MINSA (minsa): Acceso de solo lectura para extraer reportes oficiales de ingresos, egresos y jornales.
4. Cliente / Vecino (cliente): Visualiza su estado de riesgo en lenguaje amigable, verifica puntos acumulados por gamificación, revisa su historial y confirma asistencia a jornales.
5. Soporte Técnico (dev): Lectura del log de auditoría del sistema para depuración (sin acceso a modificación de finanzas).

7. Entorno operativo

- Arquitectura: PWA (Progressive Web App) alojada en la nube (ej. Vercel o GitHub Pages) con arquitectura Offline-First.
- Cliente Web/Móvil: React 18, Vite, compatible con navegadores modernos (Chrome, Firefox, Safari) en dispositivos móviles de gama media/baja.
- Base de Datos Local: IndexedDB manejada mediante Dexie.js (capacidad de hasta 10,000 registros).
- Base de Datos Remota: PostgreSQL a través de Supabase.

8. Requerimientos funcionales

8.1. Gestión de Usuarios y Accesos

- **RF-01.1**: El sistema debe registrar vecinos como “pendientes” hasta su aprobación por el Administrador.
- **RF-01.2**: El sistema debe manejar 5 roles estrictos (admin, cobrador, minsa, cliente, dev).

8.2. Pagos Flexibles y Offline

- **RF-02.1**: El sistema debe registrar pagos estándar (\$4.00 netos: \$3.00 tarifa + \$1.00 comisión), parciales, anticipados y diarios; con soporte de almacenamiento local en escenarios sin conectividad a la red.

- **RF-02.2:** El sistema debe calcular automáticamente el estado del vecino (Activo, Moroso, Para Corte) basándose en meses de atraso.

8.3. Comisiones y Recompensas (Gamificación)

- **RF-03.1:** El sistema debe otorgar puntos (ej. 5 por puntualidad, 8 por jornal personal) a los clientes.
- **RF-03.2:** El sistema debe permitir canjear puntos (ej. 1 pt = \$0.10) en futuros cobros.
- **RF-03.3:** El sistema debe calcular el “Split de Comisiones” para el cobrador (ej. 40% del monto base de comisión) de forma atómica.

8.4. Inteligencia Artificial y Análisis

- **RF-04.1:** El módulo de IA debe asignar un Puntaje de Riesgo (0-100) a cada hogar.
- **RF-04.2:** El sistema debe generar una “Ruta Inteligente” que reordene la lista de visitas del cobrador priorizando riesgo y proximidad geográfica.
- **RF-04.3:** El sistema debe predecir y alertar hogares que caerán en morosidad mediante análisis Z-Score de la tasa de recaudo.

8.5. Reportes MINSA y Gastos

- **RF-05.1:** Exportación trimestral de ingresos, egresos y jornales a PDF (jspdf) y Excel (xlsx).

9. Reglas de negocio

- **RN-01:** La sincronización hacia Supabase debe priorizar el Timestamp local. En caso de conflicto de red, el registro local marcado como ‘pendiente’ sobrescribe el remoto si su marca de tiempo es mayor.
- **RN-02:** Las multas por inasistencia a un jornal (\$10.00) se aplican automáticamente a la cuenta del vecino si no asiste y no envía sustituto.
- **RN-03:** El porcentaje del split de comisión del cobrador nunca puede afectar el monto bruto que entra a la caja de la JAAR; se calcula como un incentivo separado sobre un fondo asignado o un recargo de conveniencia.
- **RN-04:** Un cliente en estado “Para Corte” (3+ meses de morosidad) solo retorna a “Activo” a través de una operación de “Puesta al Día” completa.

10. Requerimientos de interfaces externas

10.1. Interfaces de usuario

- Diseño Mobile-First: Botones de gran tamaño para facilitar la operación en campo bajo alta luminosidad (sol directo).
- Semáforo de Estados: Colores estandarizados para morosidad (Verde=Bajo Riesgo, Amarillo=Medio, Naranja=Alto, Rojo=Crítico).
- CSS Variables: Soporte para Dark/Light Mode y tipografía clara.

10.2. Interfaces de hardware

- (Opcional a futuro) Compatibilidad con API Web Bluetooth para impresoras térmicas de recibos POS.

10.3. Interfaces de software

- Supabase SDK (@supabase/supabase-js) para autenticación y persistencia de datos.

10.4. Interfaces de comunicación

- Uso intensivo de Service Workers para interceptar peticiones de red y cachear archivos estáticos (JS, CSS) e imágenes, permitiendo que el PWA se instale en la pantalla de inicio del dispositivo.

11. Requerimientos no funcionales

- **RNF-01 (Offline):** La aplicación debe funcionar ininterrumpidamente sin conexión; operaciones locales CRUD en Dexie.js deben tomar < 500ms.
- **RNF-02 (Rendimiento):** El TTI (Time To Interactive) de la aplicación debe ser inferior a 3 segundos en conexiones 3G.
- **RNF-03 (Seguridad):** Uso estricto de Row Level Security (RLS) en Supabase para impedir que los cobradores manipulen el libro mayor de otros cobradores o las finanzas históricas.
- **RNF-04 (Privacidad):** Un vecino (cliente) en ninguna circunstancia puede acceder a las deudas o puntajes de riesgo de otros vecinos.

12. Otros requerimientos

- Los datos (exportaciones XLSX/PDF) deben generarse al lado del cliente (Client-Side Generation) para evitar carga en el servidor y permitir generación offline.

13. Posicionamiento

13.1 Oportunidad de Negocio

Aproximadamente 3,682 JAAR legalmente constituidas en Panamá manejan finanzas en libretas susceptibles a pérdida, humedad y fraudes, generando desconfianza comunitaria.

13.2 Sentencia que define el problema

La falta de conectividad en zonas rurales imposibilita el uso de ERPs modernos, y el registro manual genera morosidad prolongada y penalizaciones por parte del MINSA ante la falta de reportes claros.

13.3 Sentencia que define la posición del Producto

Para las Juntas de Acueductos Rurales (JAAR) e instituciones públicas (MIDES, MINSA), SIMAP Digital es la única Plataforma Web Offline-First que asegura recaudo eficiente con IA, fomenta la puntualidad mediante puntos, y garantiza la auditoría estatal instantánea.

14. Descripción de Stakeholders y Usuarios

14.1 Resumen de Stakeholders

- MINSA — División de Saneamiento Rural: Ente regulador. Recibe reportes trimestrales (Excel/PDF) para supervisar el cumplimiento del Decreto Ejecutivo N° 1839.
- Directiva de la JAAR: Líderes electos y patrocinadores. Necesitan un sistema que controle pagos, jornales y gastos para asegurar la transparencia y reducir conflictos por morosidad.
- Desarrollador del Sistema: Arquitecto de software y mantenedor. Busca que el sistema funcione establemente y sirva como modelo replicable para otras JAAR.

14.2 Resumen de Usuarios

- Cobradores / Tesoreros: Usuarios operativos en campo que registran cobros y gastos.
- Moradores / Vecinos: Usuarios finales que revisan su historial de pagos y avisos de jornales.
- Administrador del Sistema: Técnico designado que gestiona los accesos y aprueba usuarios.

14.3 Entorno de usuario

Trabajo en campo (ej. Comunidad de Caballero, Antón) mediante dispositivos móviles (celulares), requiriendo funcionamiento sin internet (offline) para el registro de datos y sincronización posterior debido a la conectividad intermitente.

14.4 Perfil de los Stakeholders

Líderes comunitarios y usuarios con necesidad de herramientas sencillas sin conocimientos técnicos avanzados; exigen paneles y aplicaciones fáciles de usar para reemplazar los registros manuales en cuadernos.

15. Descripción Global del Producto

15.1 Perspectiva del producto

Es una aplicación independiente (SPA PWA) acoplada a un modelo de Backend as a Service (Supabase) que entra en acción solo cuando la red es detectada por el Service Worker.

16. Características Secundarias

16.1 Foro de Avisos

Módulo donde la directiva informa cortes de agua, mantenimientos, o asambleas.

16.2 Módulo Dev/Auditoría

Logs inmutables de todo lo que ocurre en el sistema para trazabilidad técnica.

17. Restricciones

- Económicas: Se usan Free Tiers (Vercel, Supabase gratuito) para que el sistema sea viable sin cobros de licenciamiento mensual altos a comunidades de subsistencia.
- Memoria: IndexedDB en iOS Safari tiene cuotas de memoria estrictas, limitando el almacenamiento de blobs grandes.

18. Precedencia y Prioridad

1. Capa Offline (Dexie.js) y Módulo de Cobros Flexibles.
2. Sincronizador Bidireccional Supabase.
3. Reportes JS PDF y SheetJS.
4. Módulo "Ruta IA" y Gamificación.

19. Otros Requisitos del Producto

- Estándares del Formato Contable MINSA.

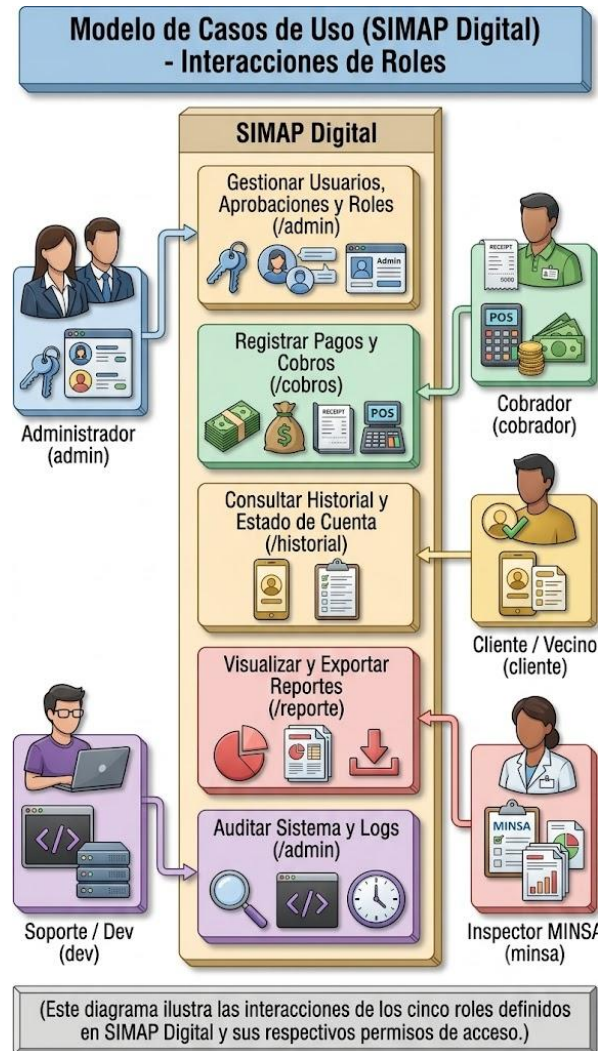
20. MODELO CASOS DE USO

20.1 Definición de Actores

- **Admin**: Configura reglas de puntos (Split), aprueba vecinos, revisa tablero de recaudo.
- **Cobrador**: Registra cobros (normales, parciales, Puesta al Día), jornales, canjea puntos, usa Ruta Inteligente.

- **Ciente:** Verifica estado de riesgo en “lenguaje amigable”, ve sus puntos, confirma asistencia.
- **Sistema (IA):** Detecta anomalías, predice morosidad, asigna puntajes de riesgo.

20.2 Diagrama de Actores



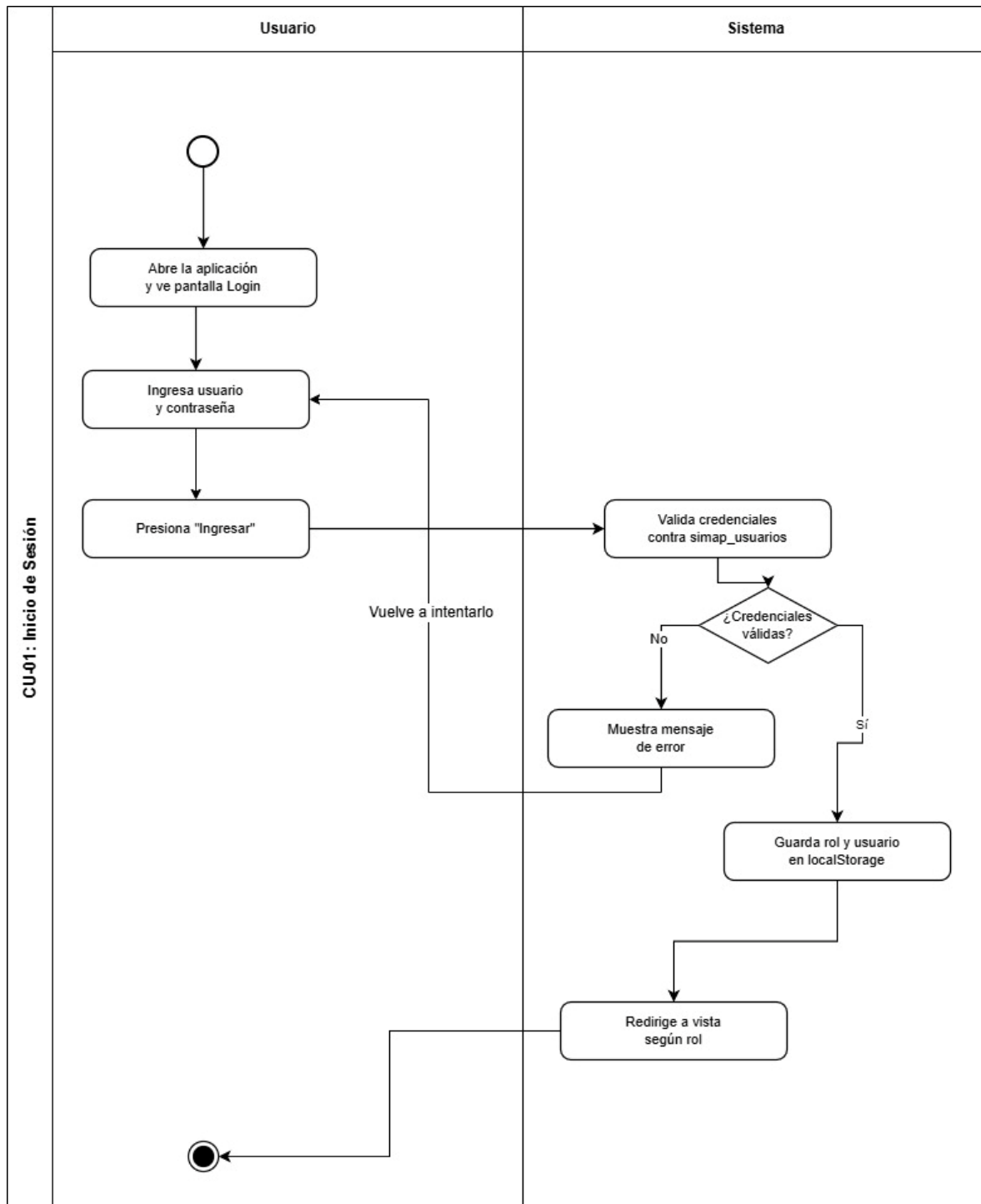
20.2.1. Casos de Uso (Extracto)

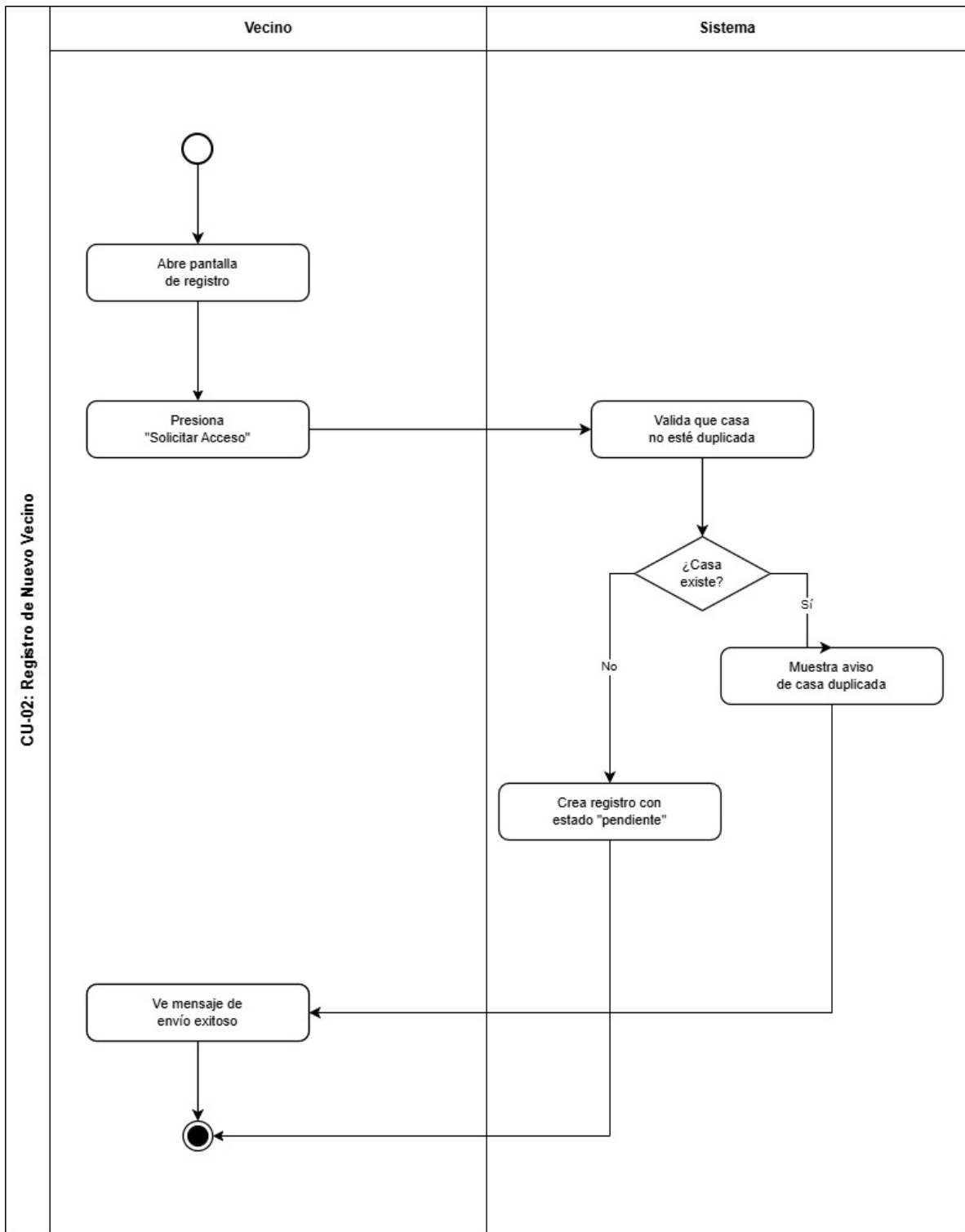
- **CU-06:** Pago Mensual Estándar.
- **CU-07:** Pago Parcial.
- **CU-11:** Cobrador Registra Pago y Recibe Comisión Automática.
- **CU-16:** Vecino Canjea Puntos por Descuento.
- **CU-19:** Cobrador Activa “Ruta Inteligente”.
- **CU-20:** Sistema Predice Hogares que Caerán en Morosidad.
- **CU-23:** Cliente Ve su Estado de Riesgo en Lenguaje Amigable.

20.2.2. Identificación de Casos de Uso

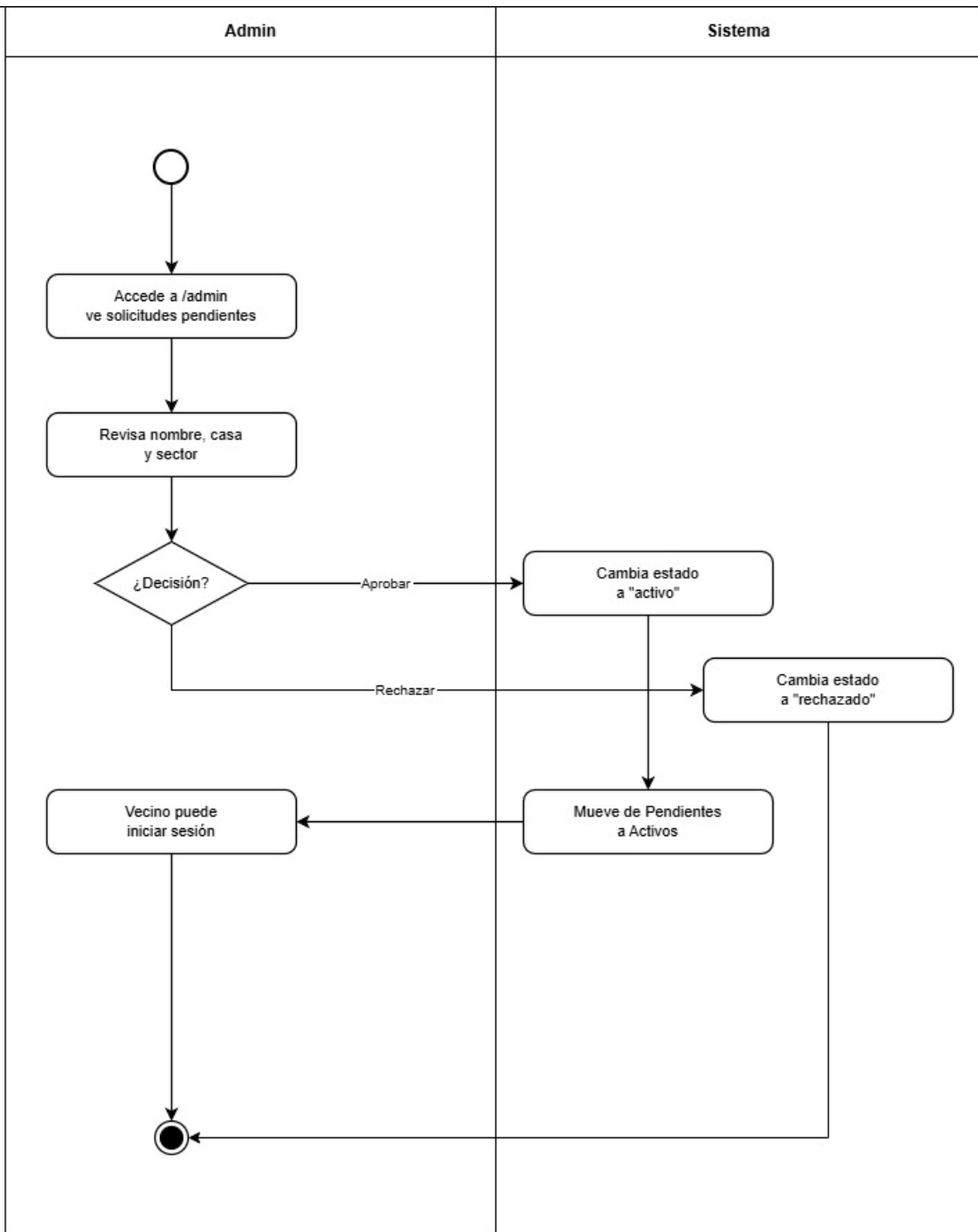
El Sistema de IA incluye (<>) la evaluación del historial del cliente de forma automatizada y el Cobrador usa el reordenamiento a voluntad.

20.2.3. Diagrama de casos de uso

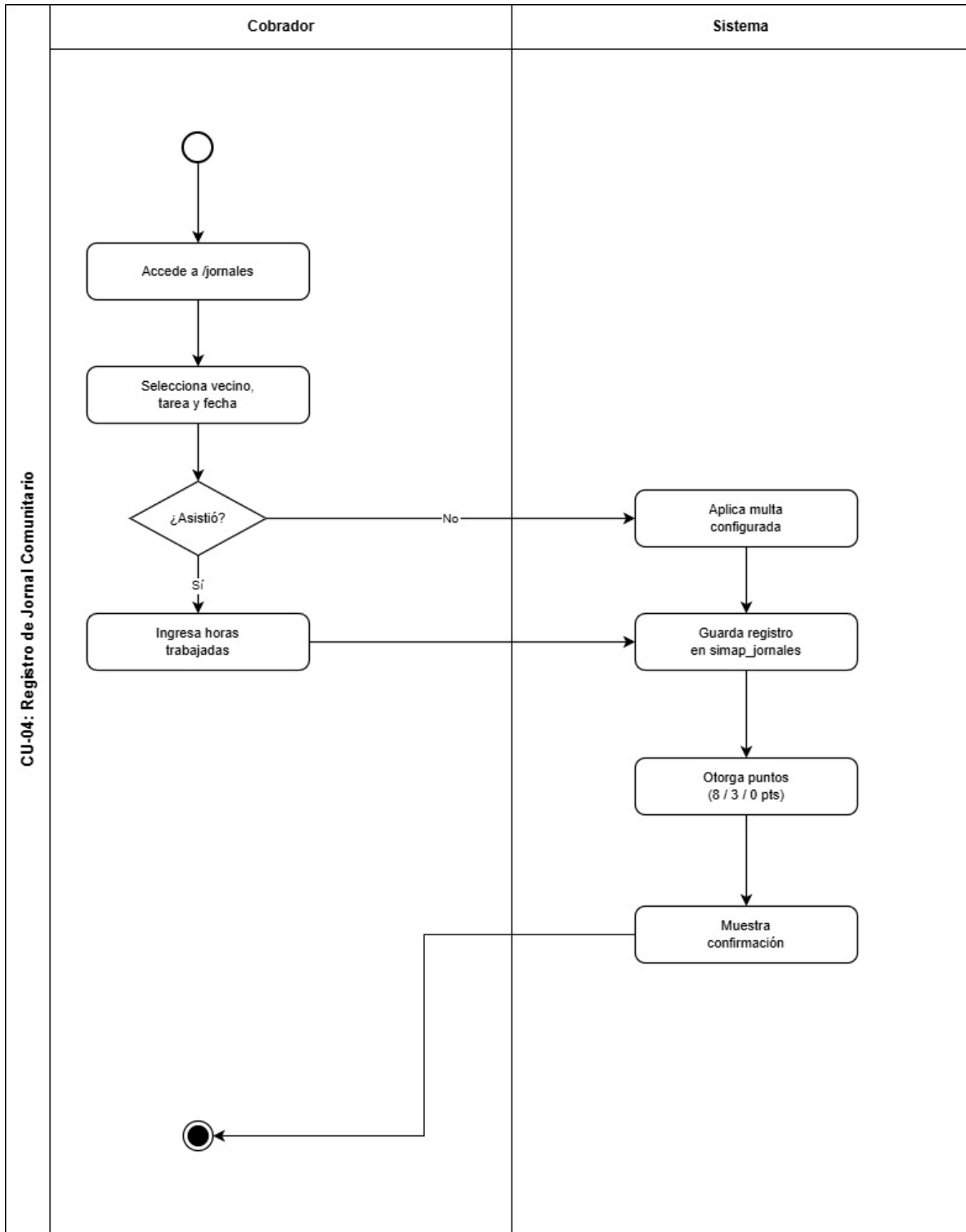




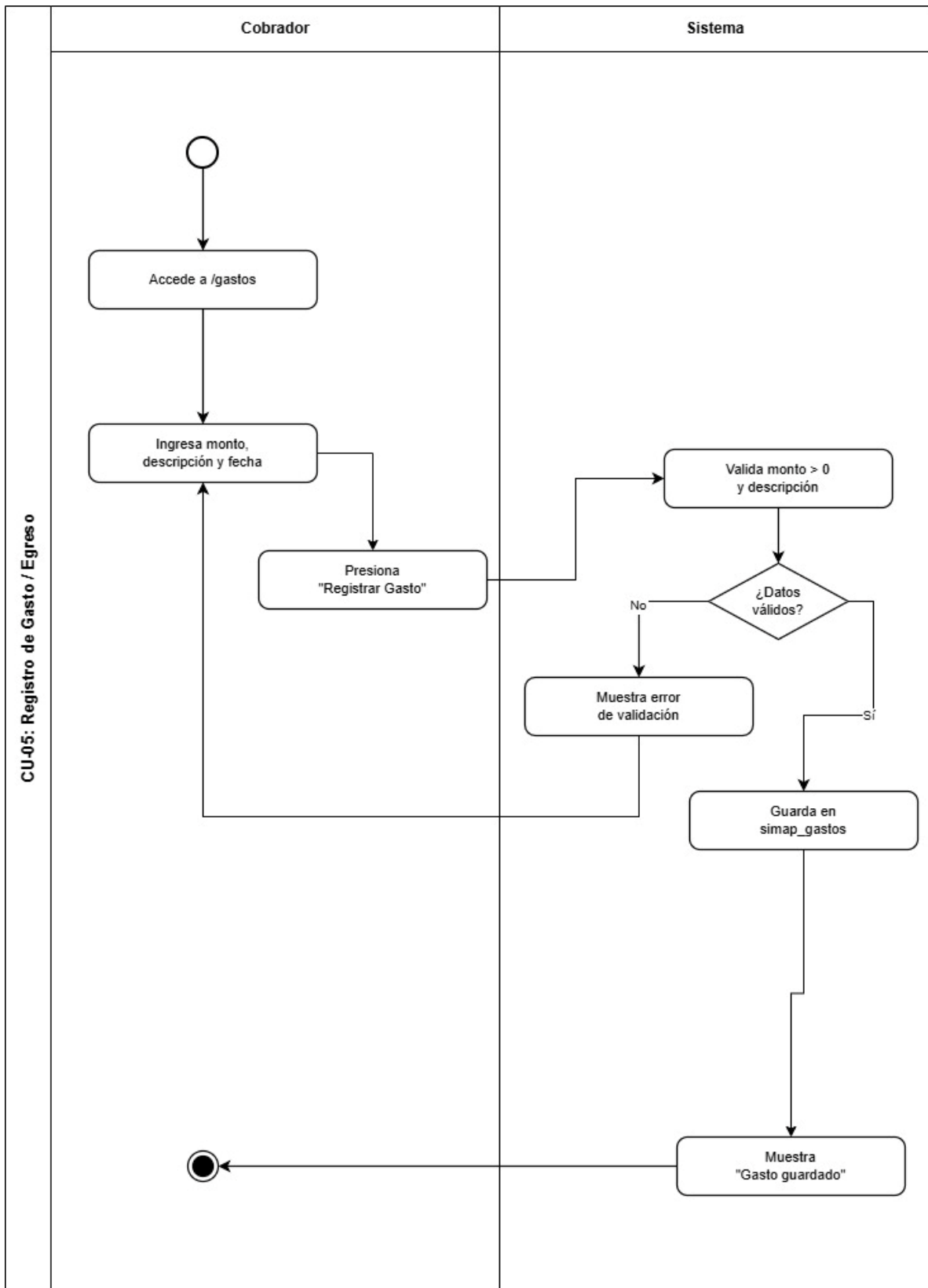
CU03: Aprobación / Rechazo de Vecino por Admin

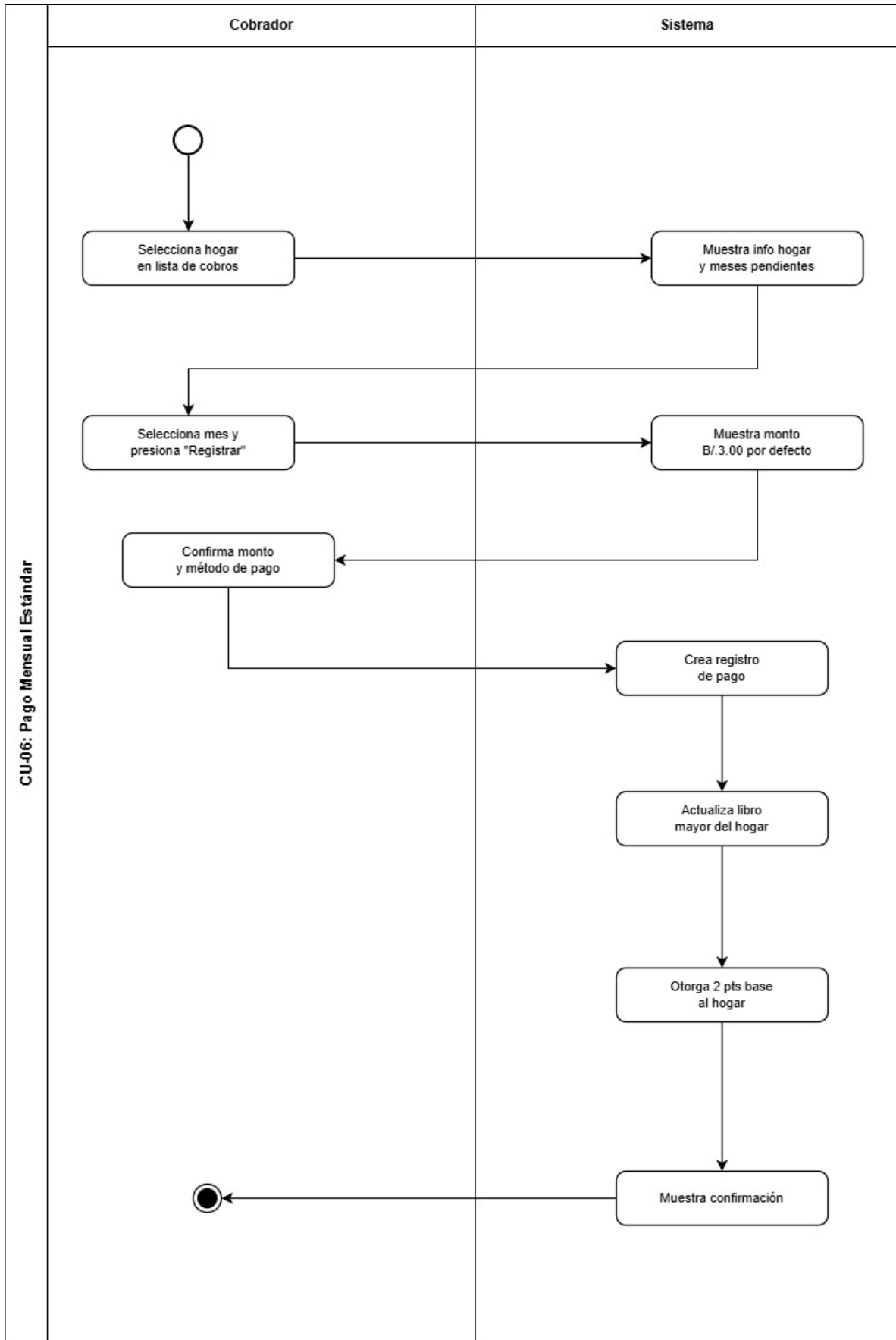


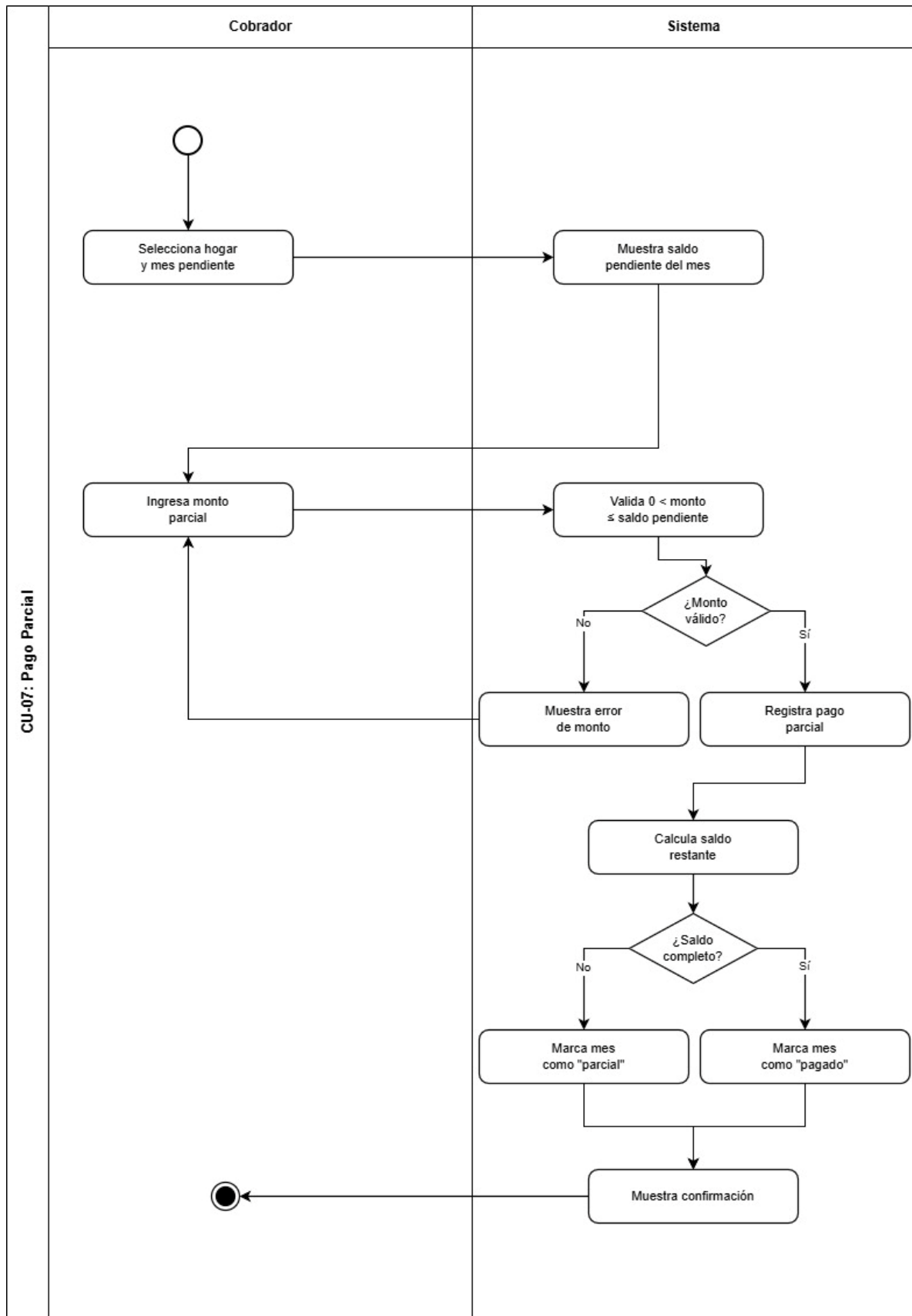
CU-04: Registro de Jornal Comunitario

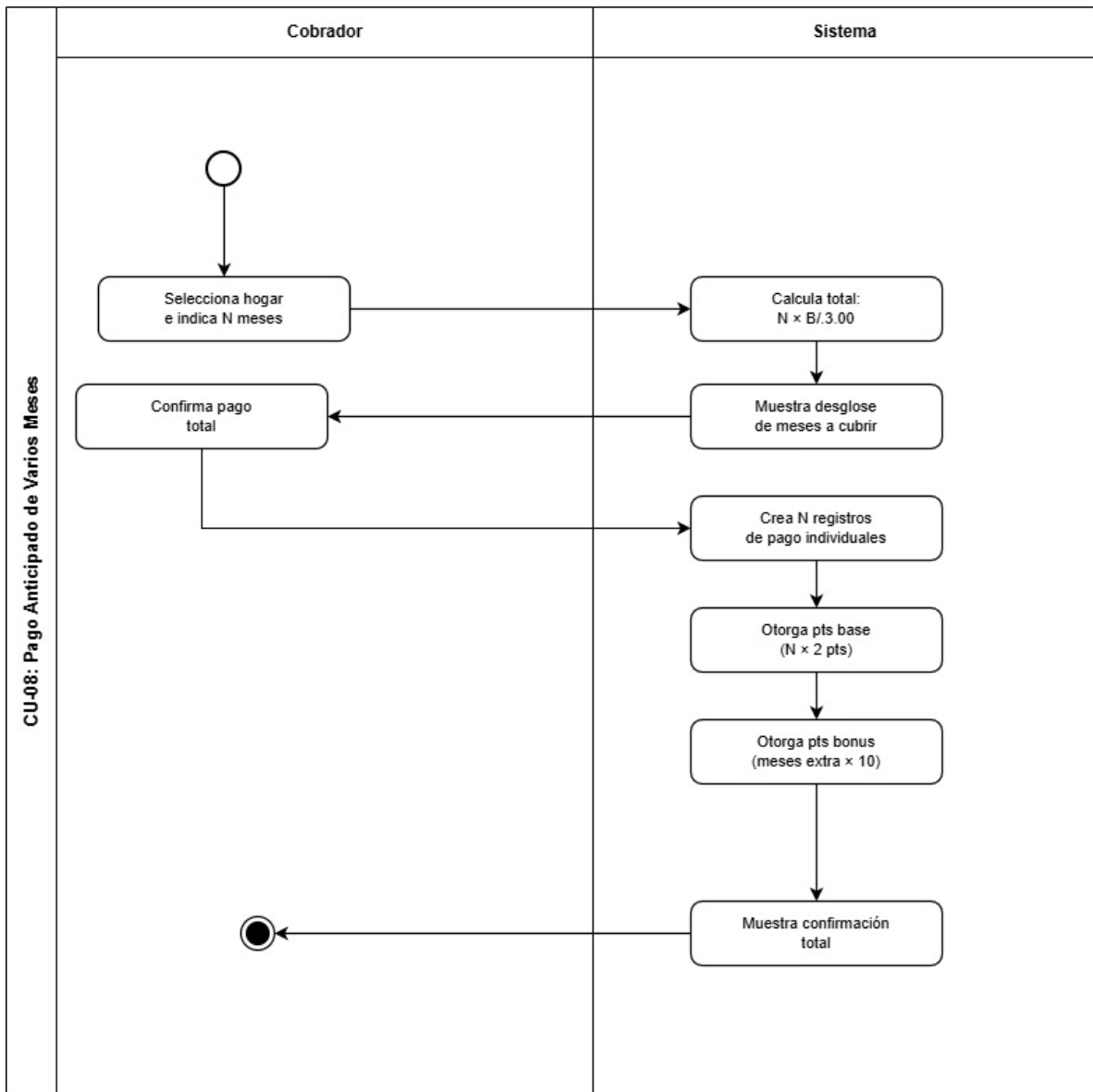


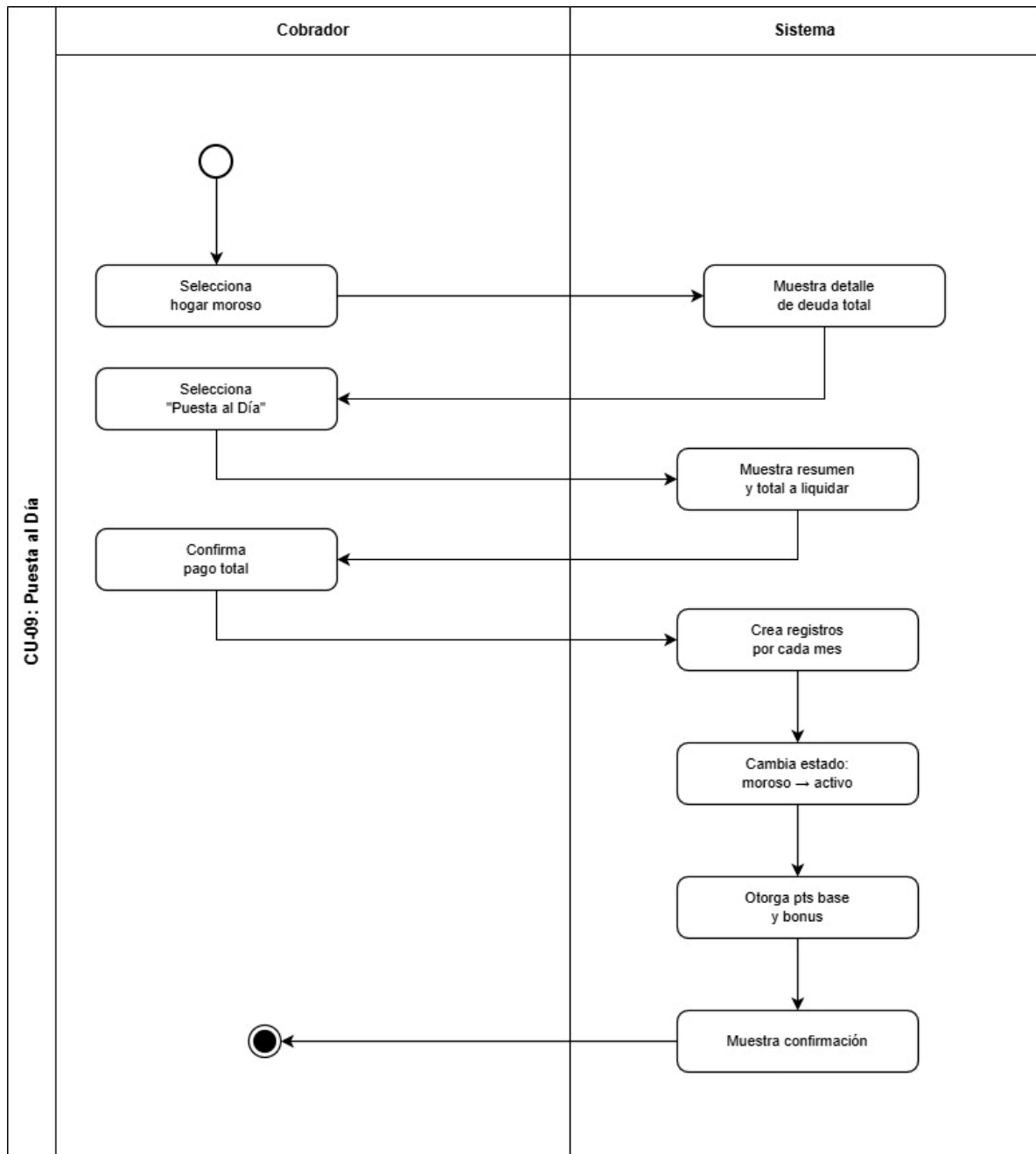
CU-05: Registro de Gasto / Egreso

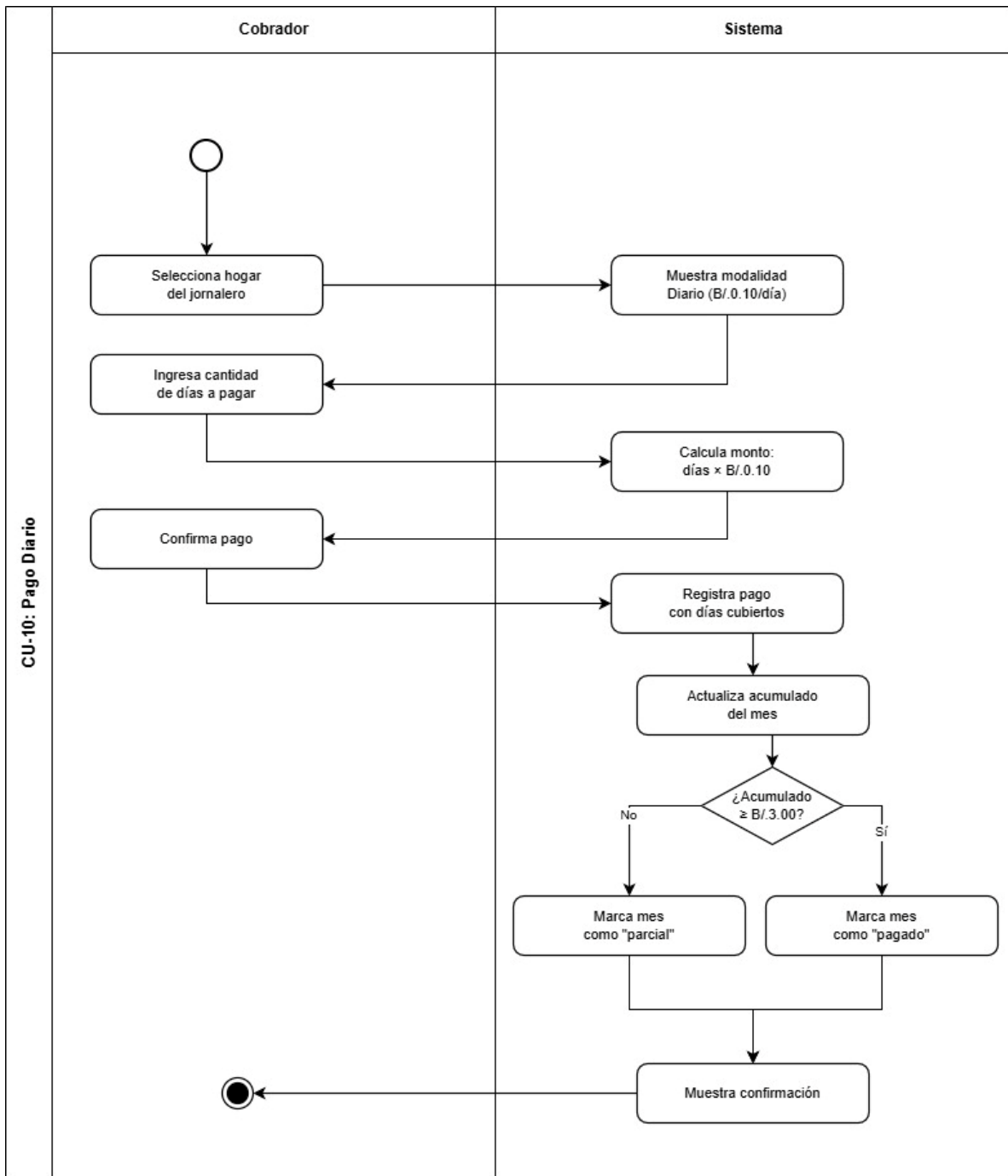


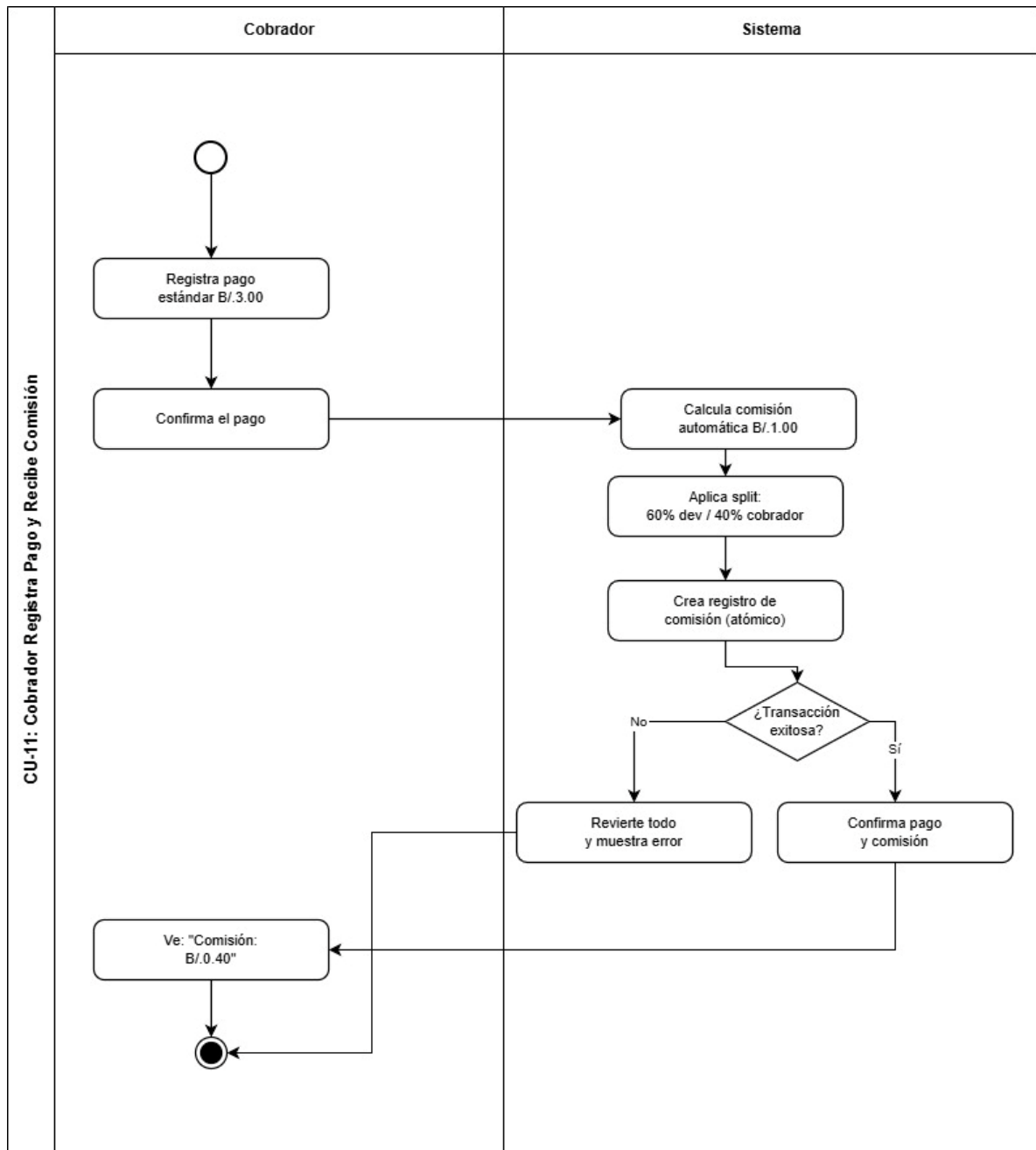


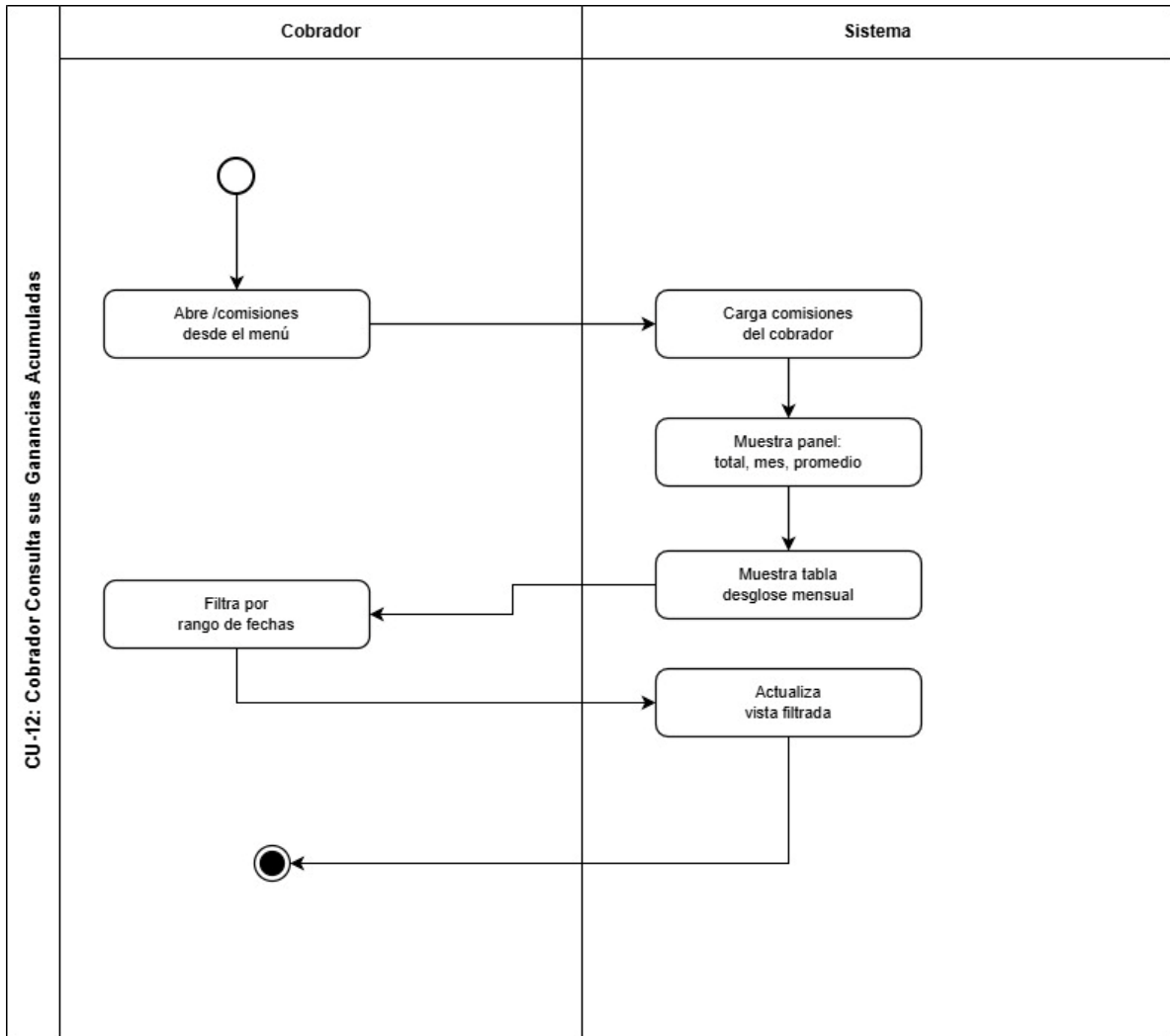




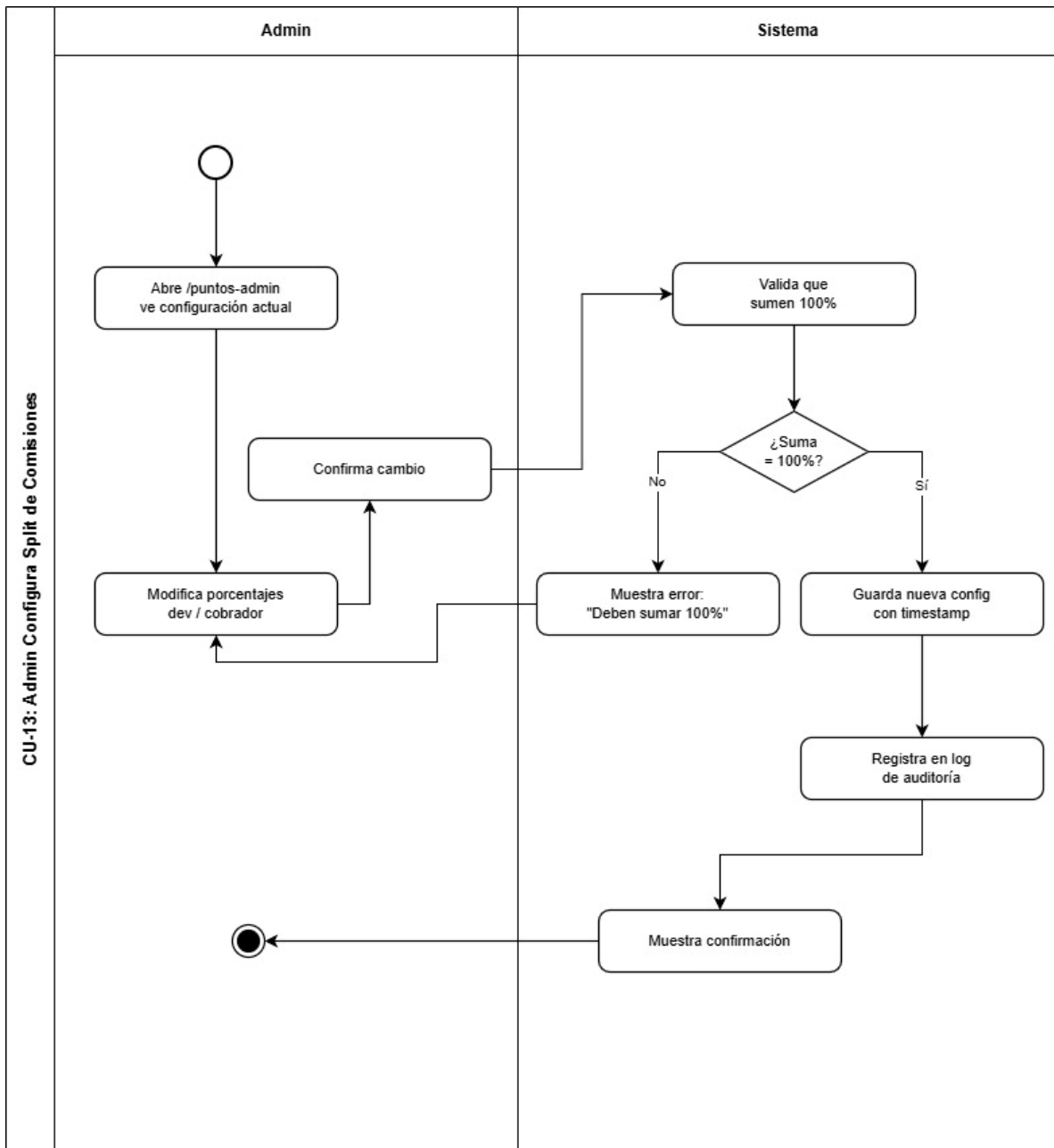






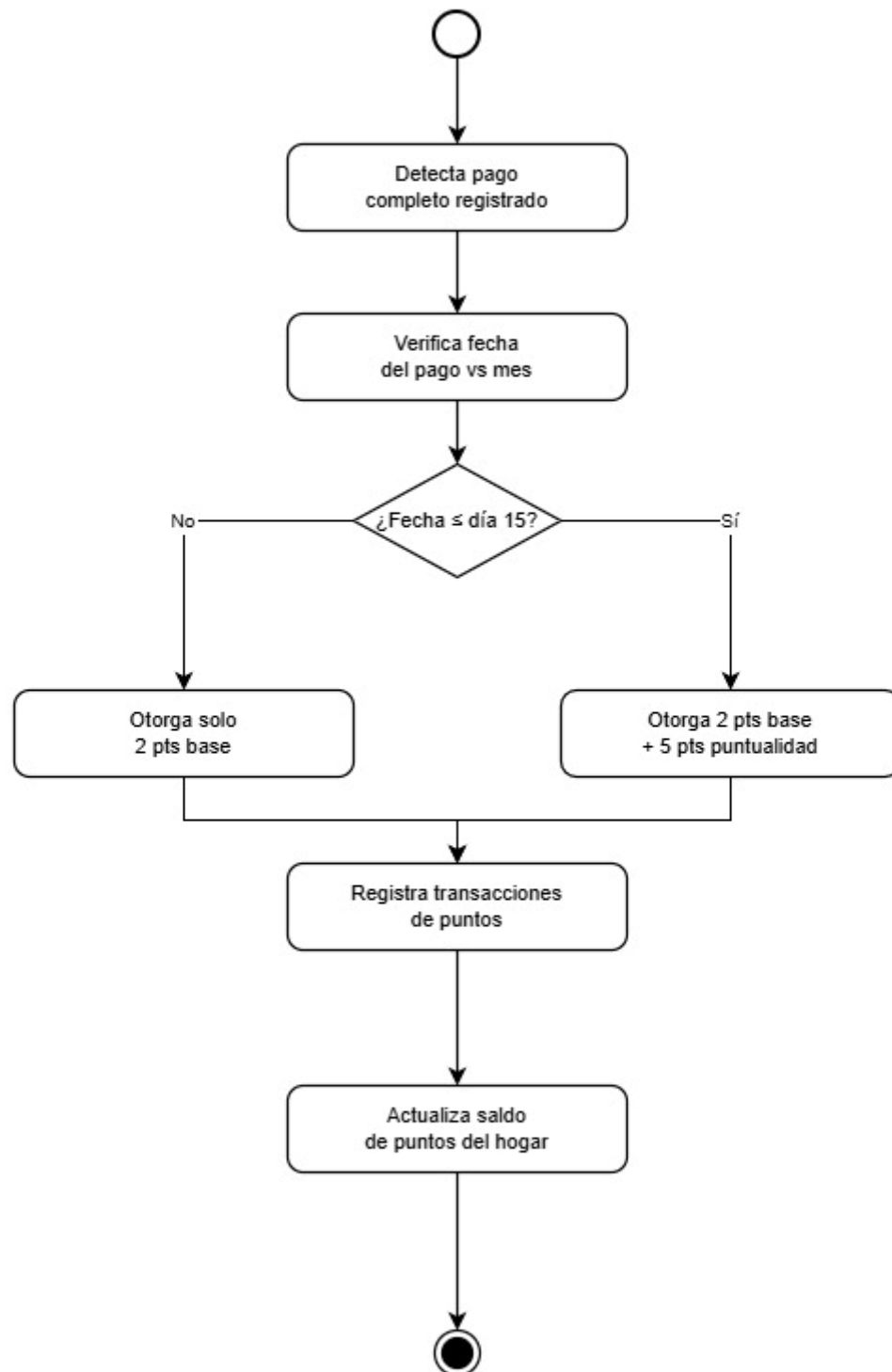


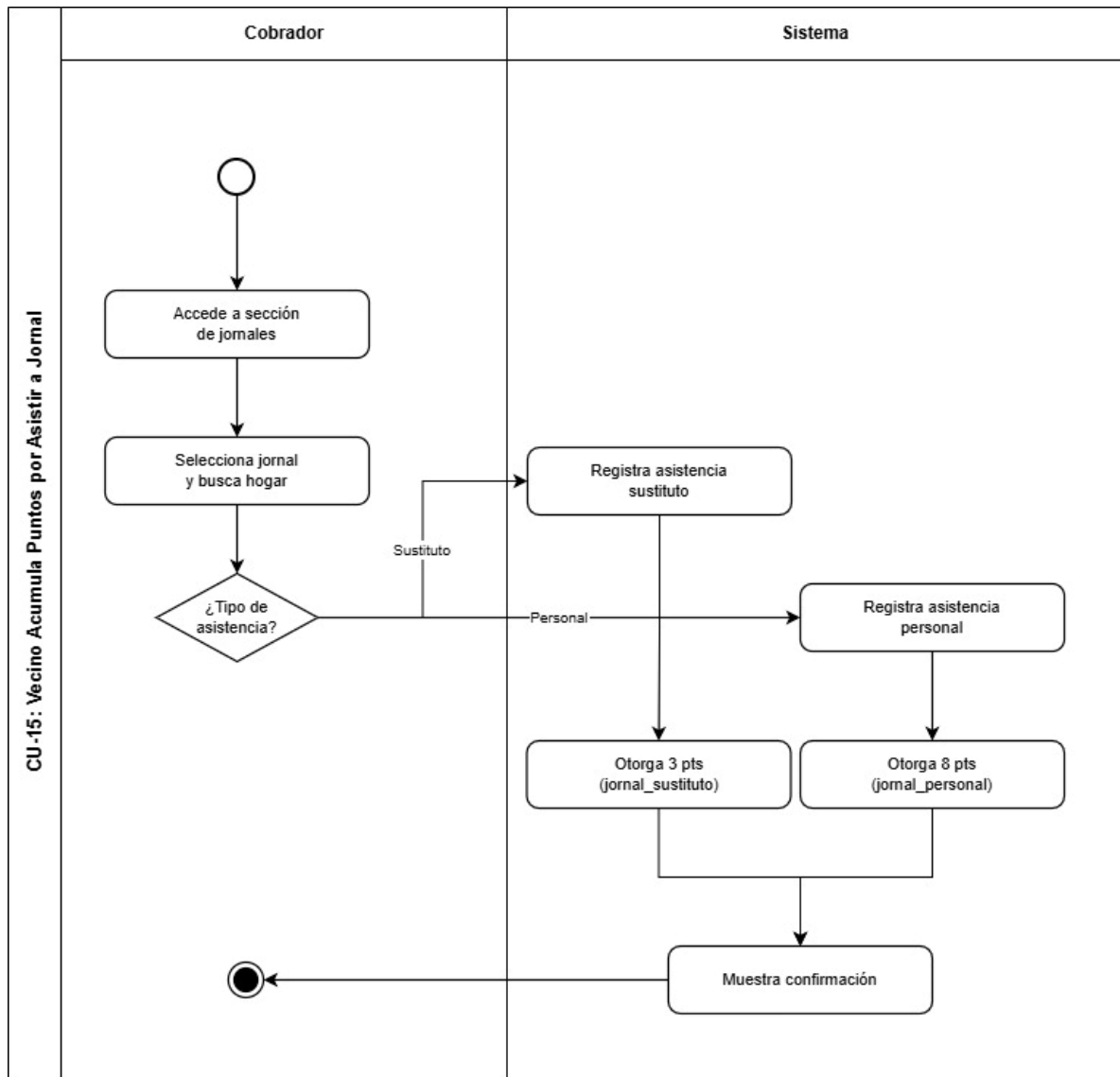
CU-13: Admin Configura Split de Comisiones

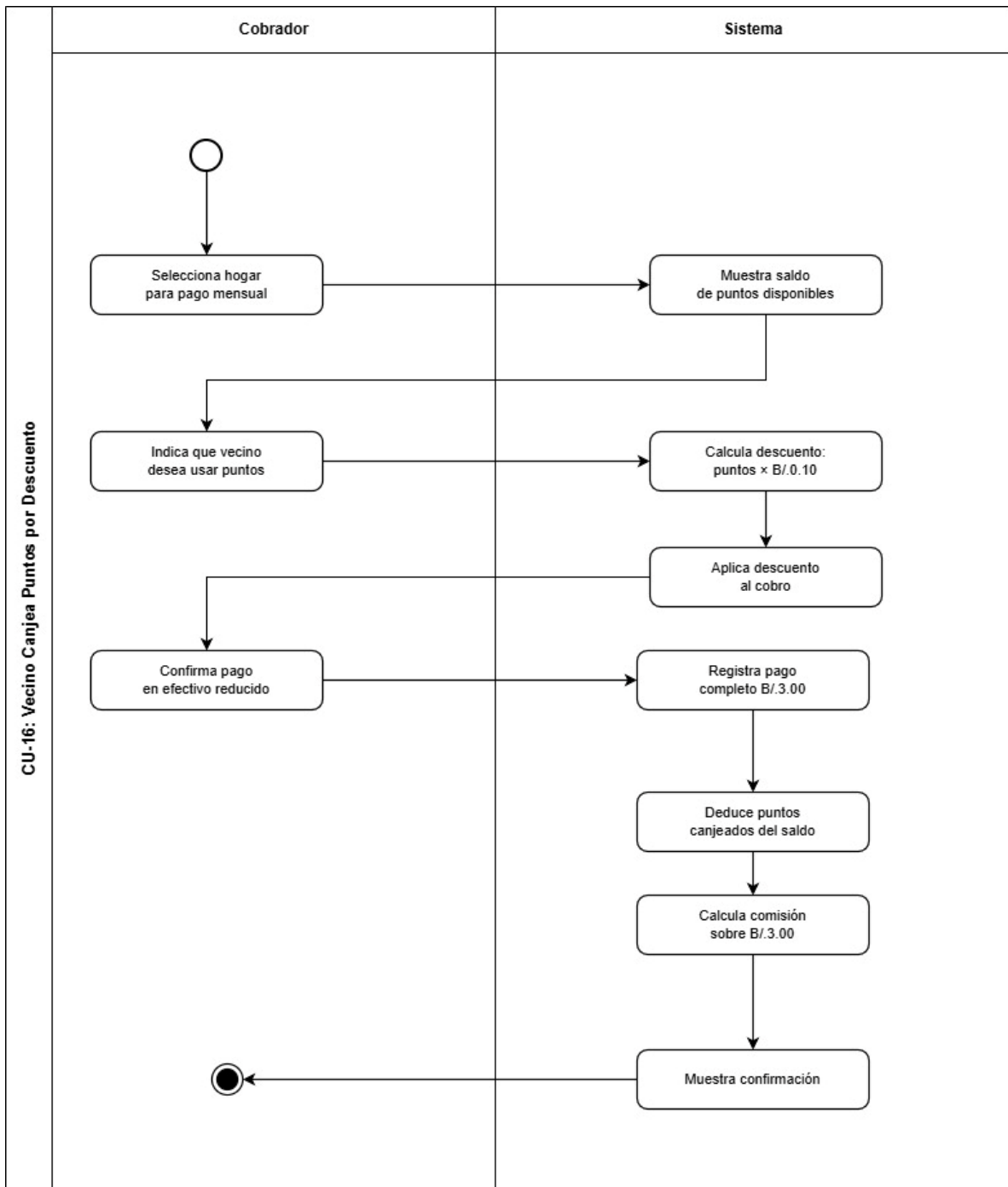


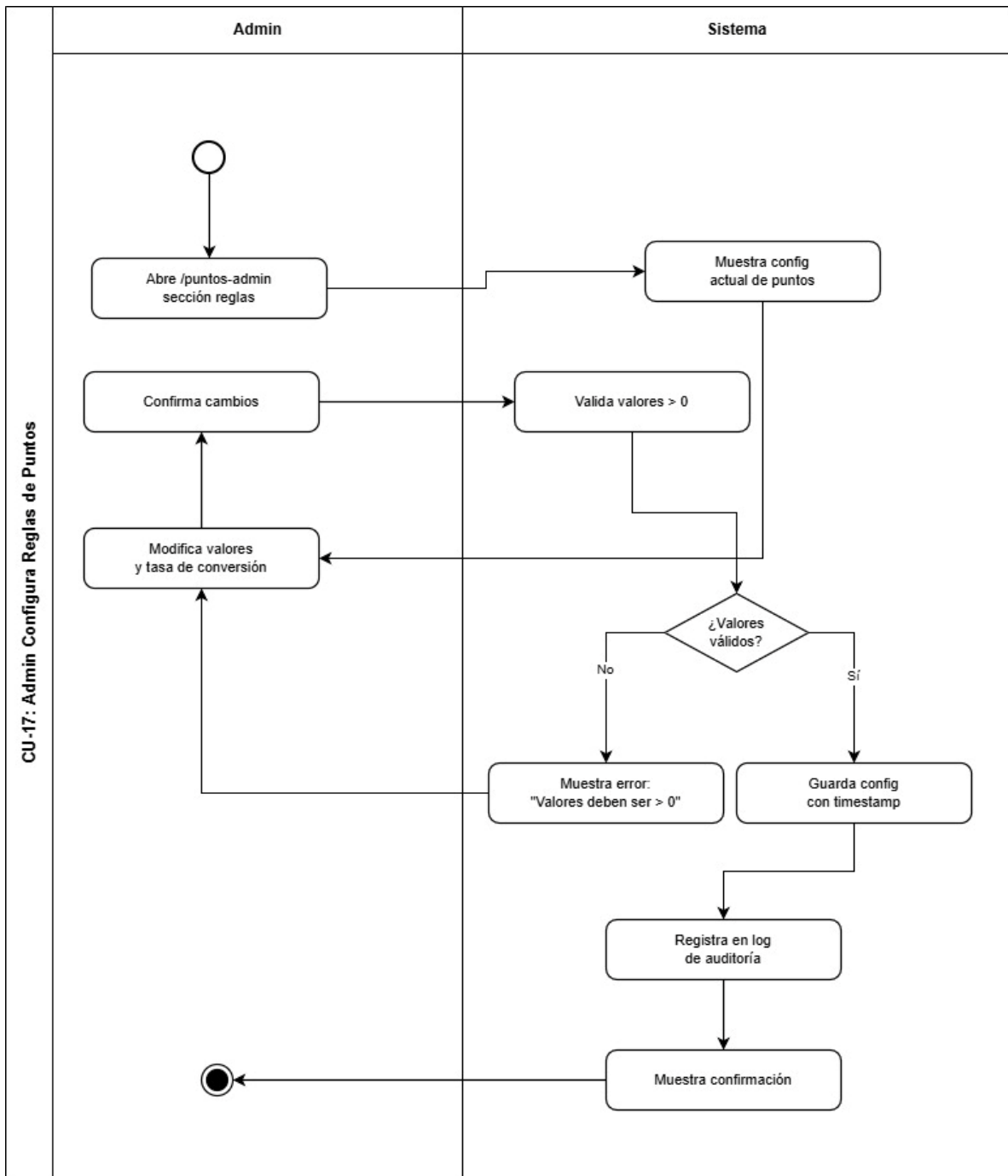
CU-14: Vecino Acumula Puntos por Pago Puntual

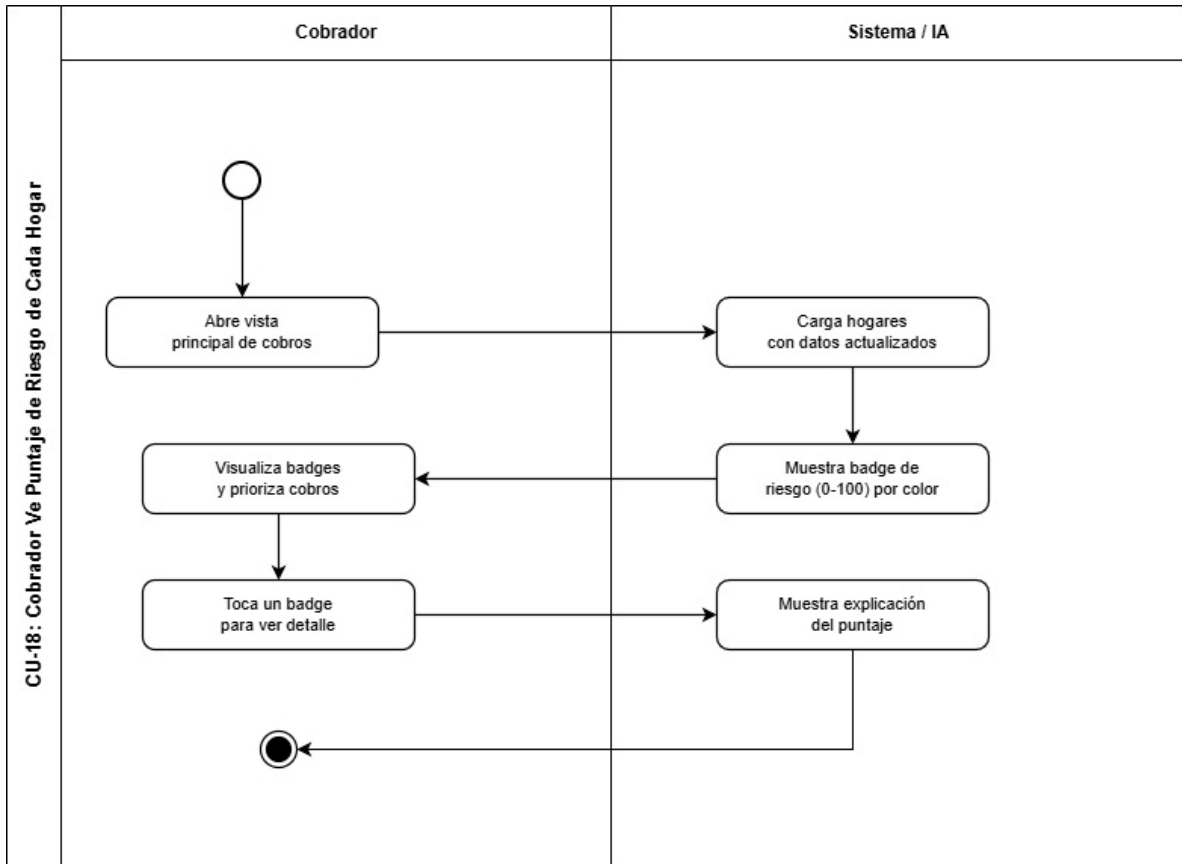
Sistema

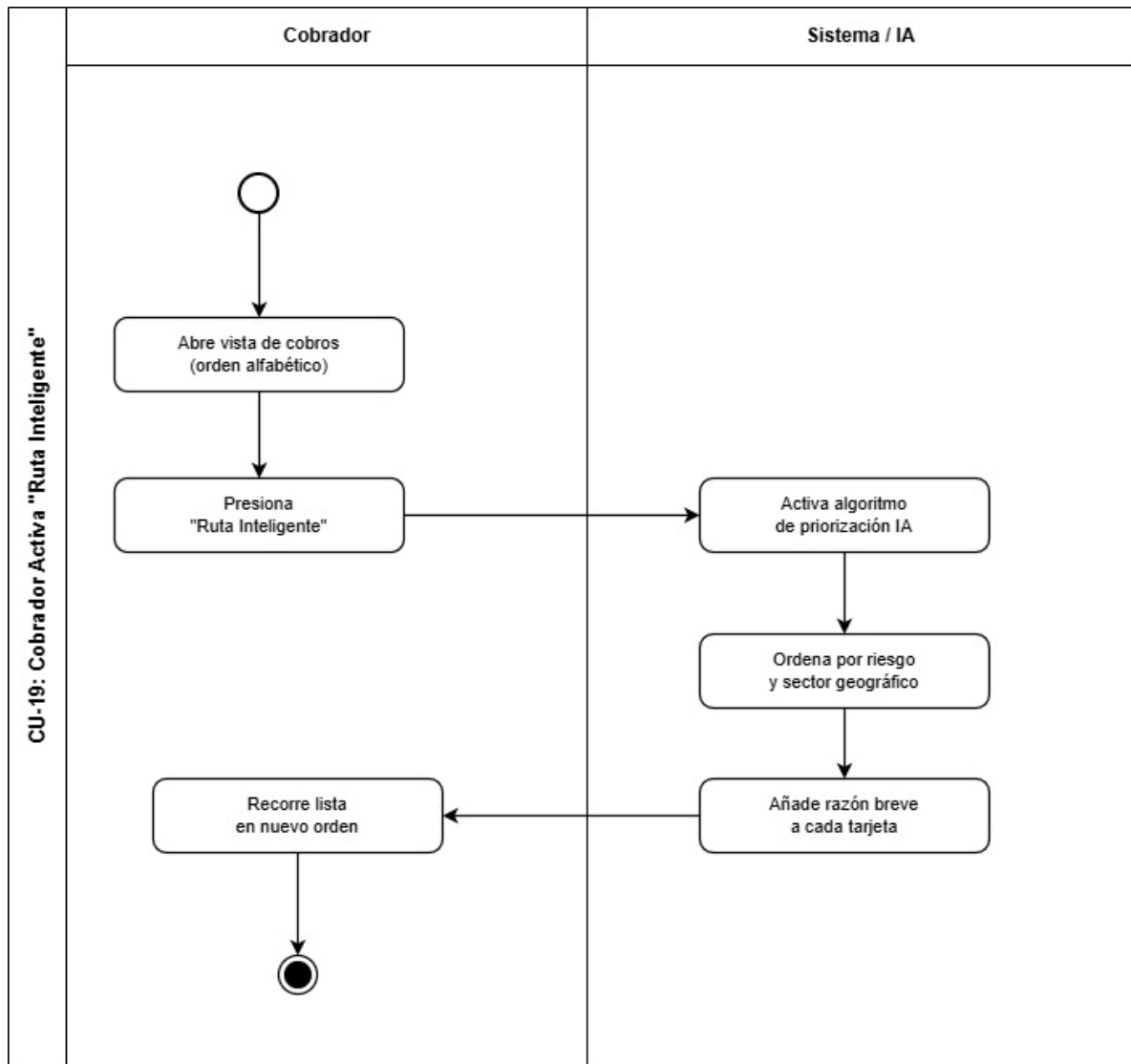


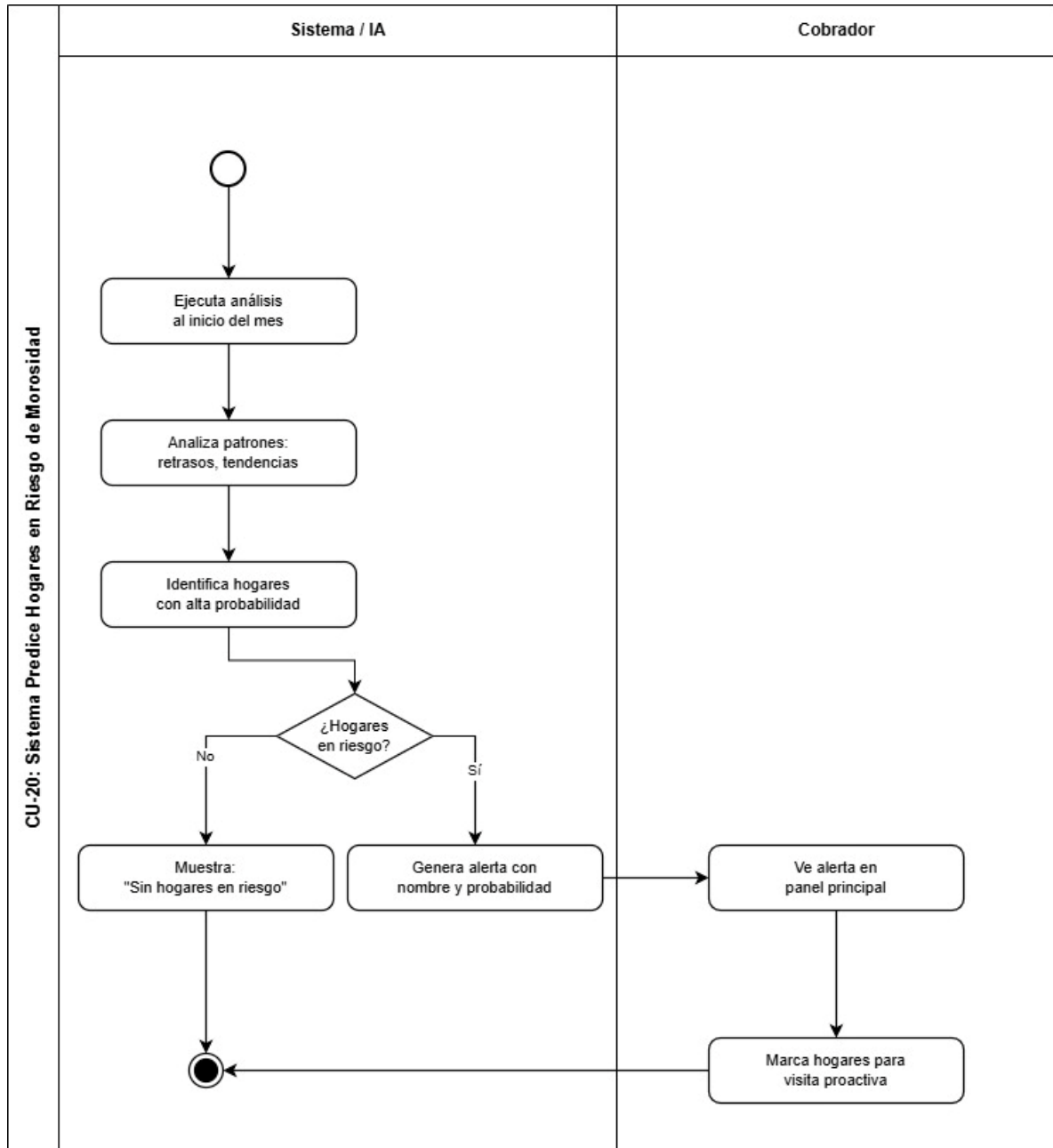


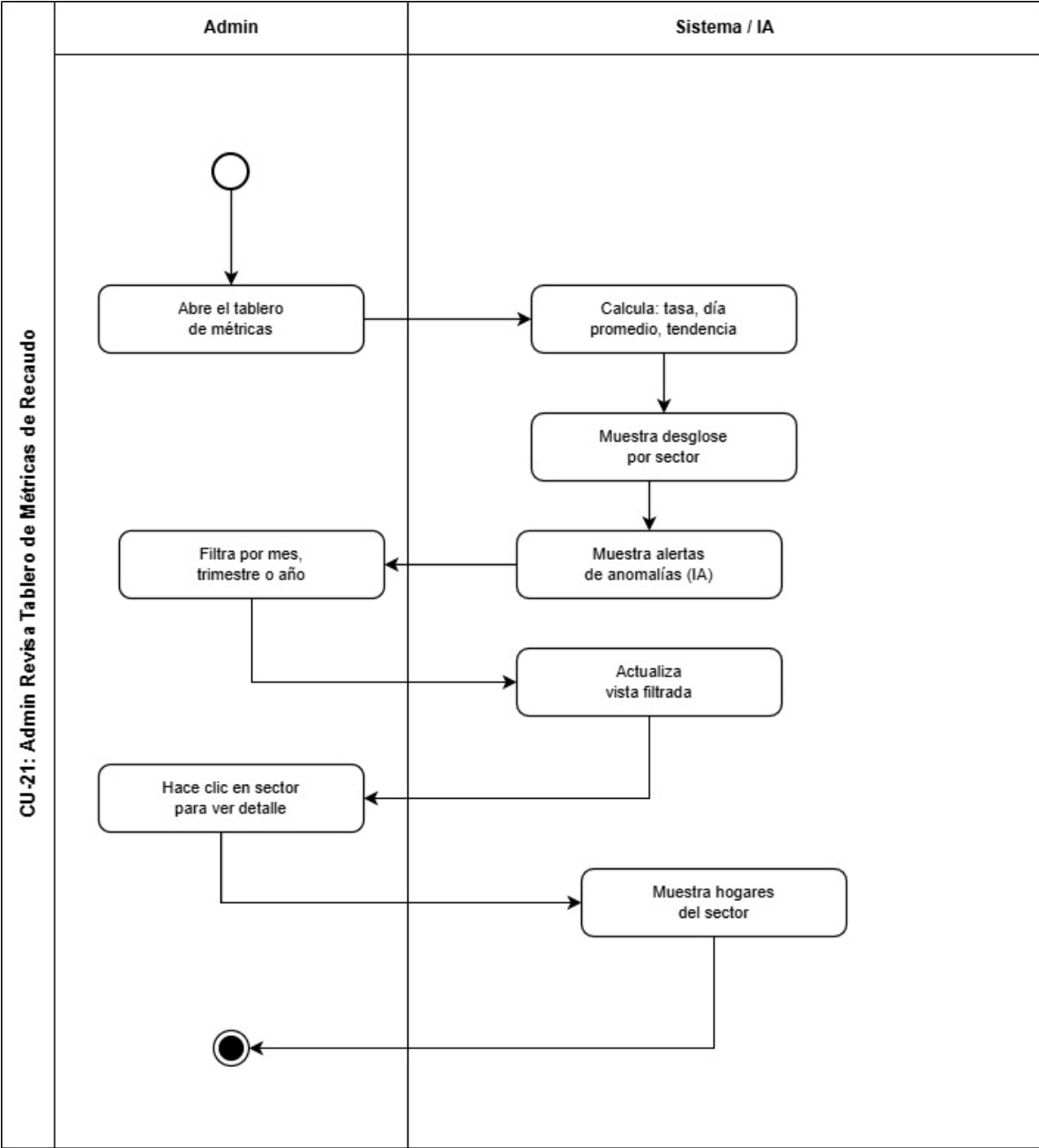


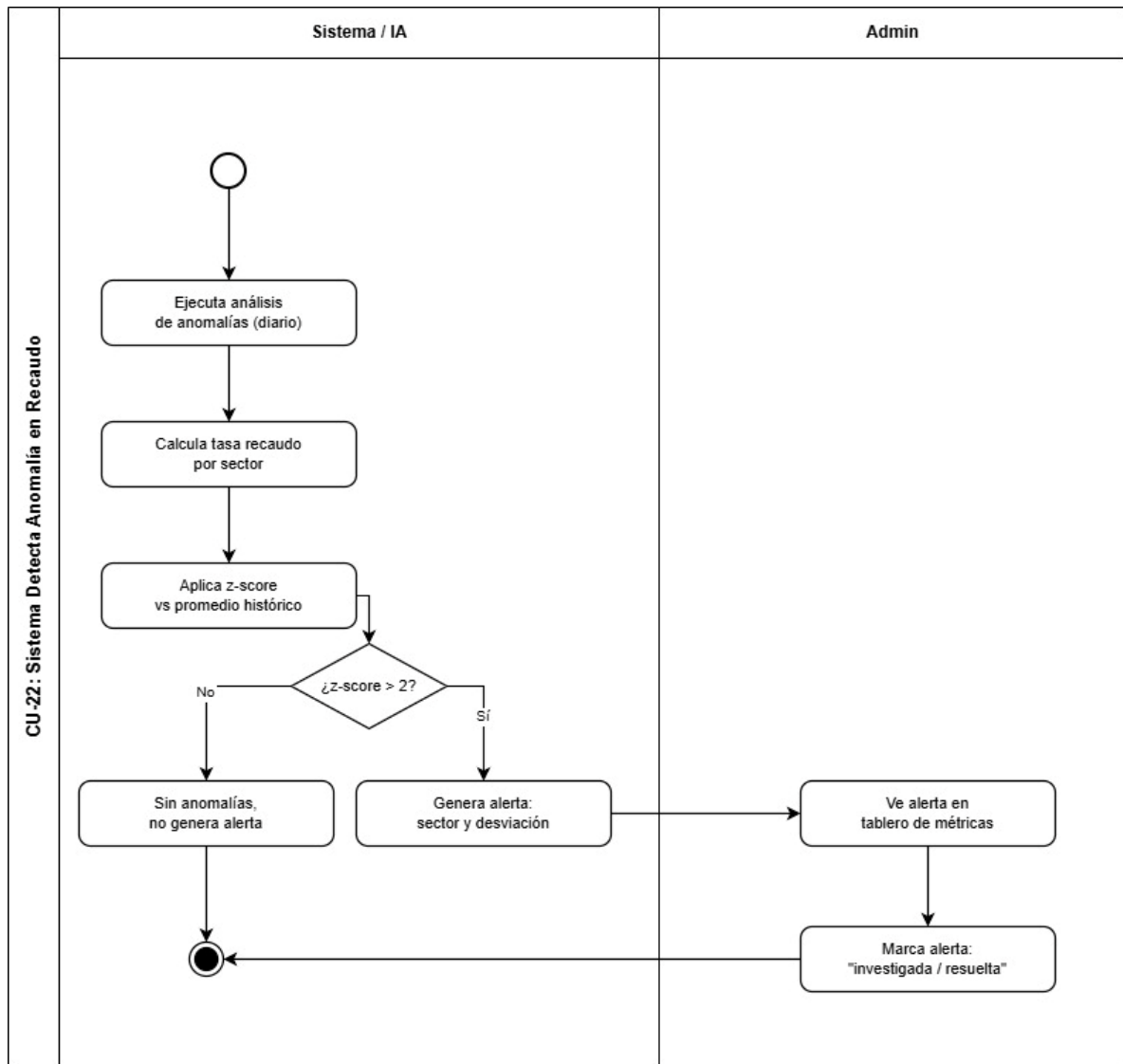


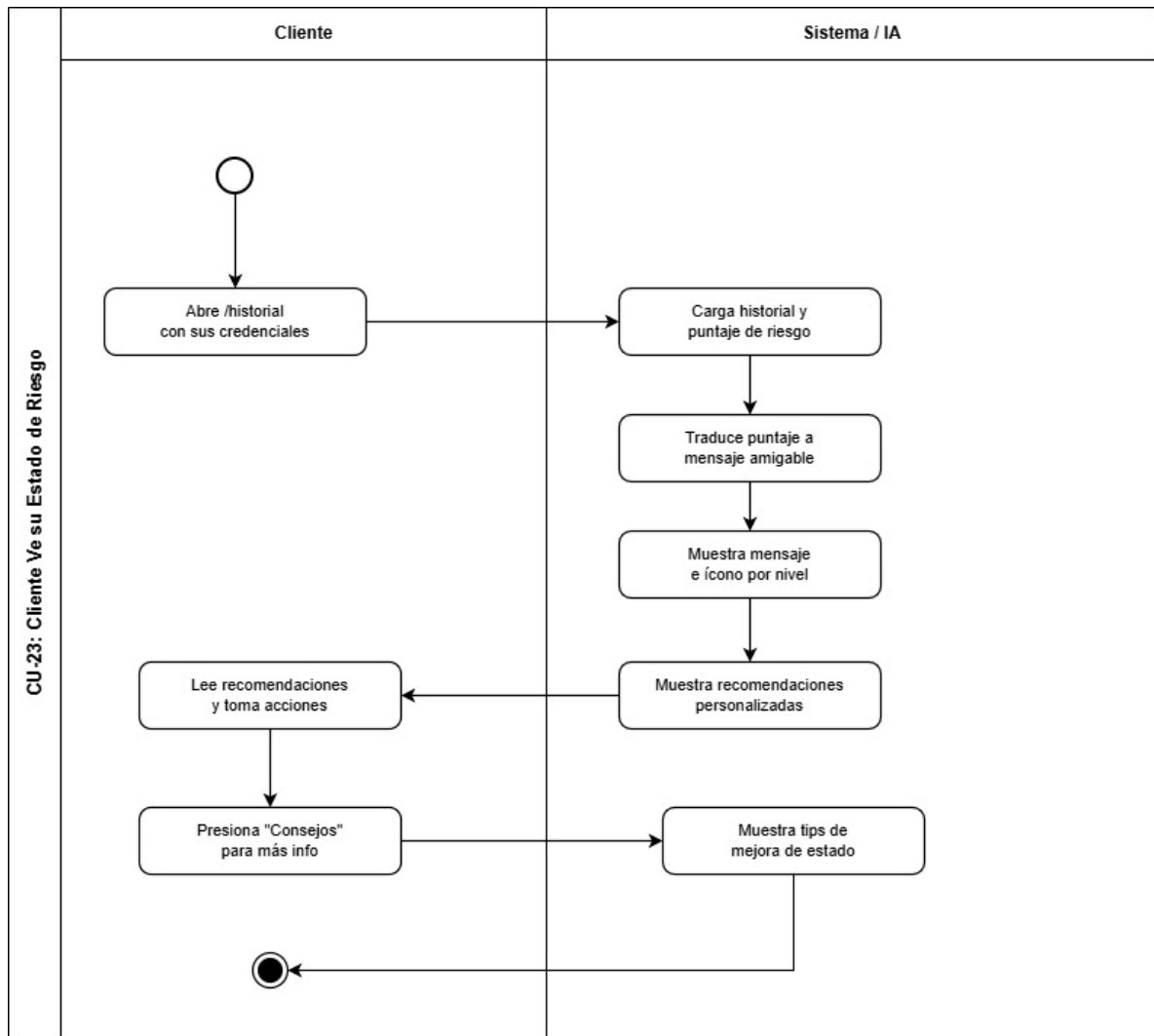






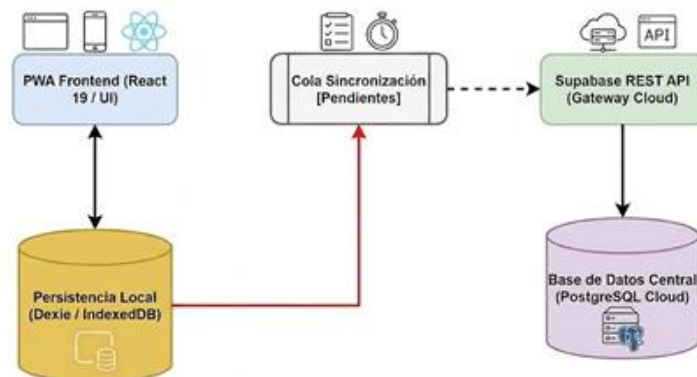






21. Modelo de Arquitectura de Análisis

21.1. Diagrama de Arquitectura de Análisis



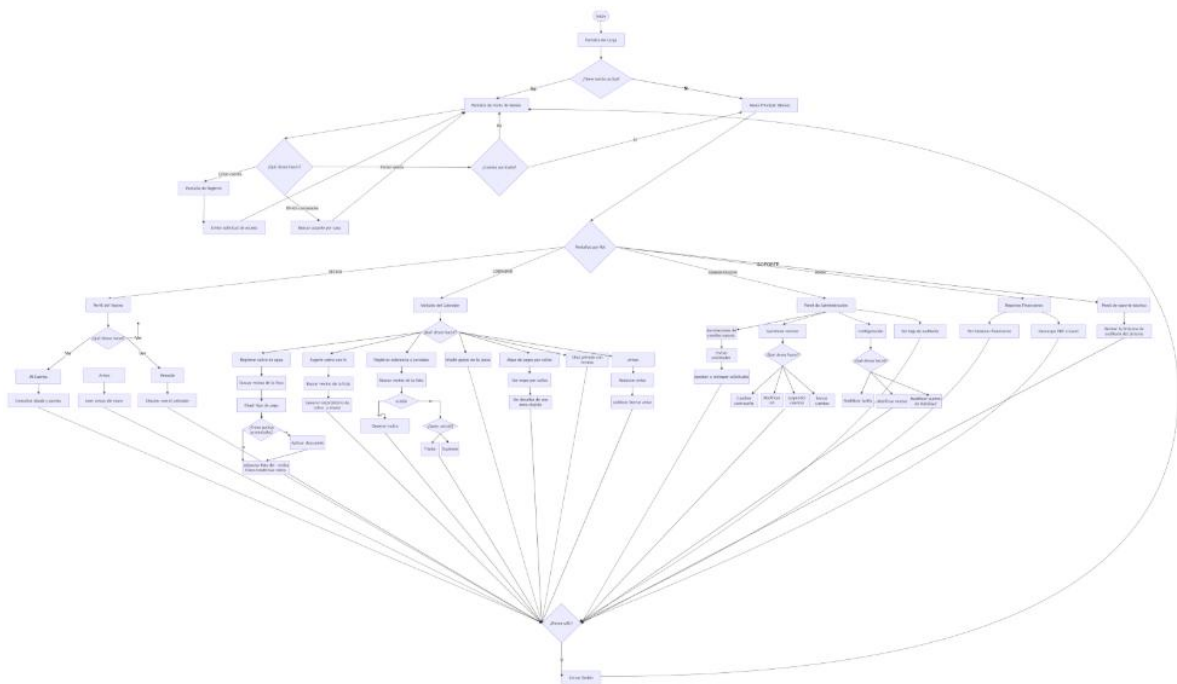
21.2. Clases de Análisis

Patrón Repository en React. Todos los componentes interactúan con servicios asíncronos (PaymentService, SyncService) que deciden si guardar en la RAM, en IndexedDB o intentar escribir directamente en la DB en Supabase.

21.3. Identificación de clases de Análisis

Se dividen en fronteras (UI), control (Context API / Custom Hooks) y entidad (IndexedDB Collections).

21.3.1. Diagrama de Secuencia



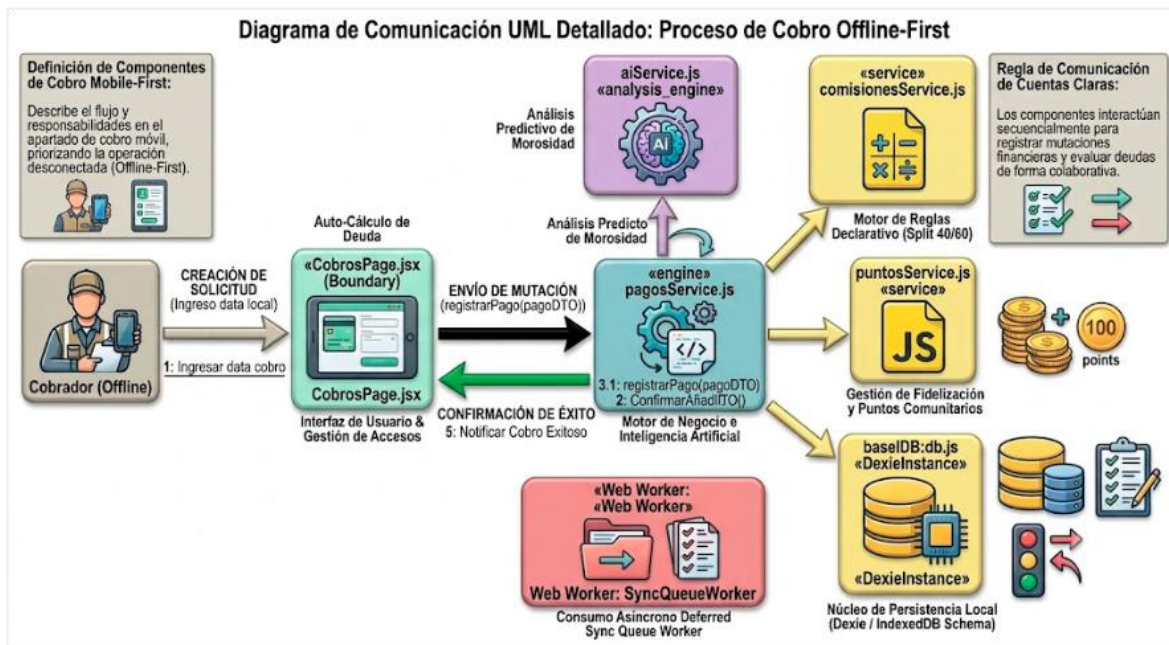
El diagrama mapea el recorrido del usuario a través de la aplicación, definiendo las siguientes rutas y validaciones:

1. Arranque y Autenticación:
 - Pantalla de Carga (Splash): Evalúa la condición ¿Tiene sesión activa?
 - Si la respuesta es Sí, entra directamente al Menú Principal. Si es No, entra a la Pantalla de Inicio de Sesión.
 - En la Pantalla de Inicio de Sesión (¿Qué desea hacer?), el usuario tiene tres opciones:
 - Crear cuenta: Lo dirige a la pantalla de registro, envía la solicitud de acceso y regresa al inicio de sesión.
 - Olvidó contraseña: Permite buscar al usuario por casa y regresa al inicio.
 - Iniciar sesión: Activa la condición ¿Cuenta aprobada? Si no está aprobada, lo devuelve al inicio; si sí lo está, accede al Menú Principal.
2. Navegación basada en Roles: Desde el Menú Principal, el flujo se bifurca dependiendo de los roles de usuario hacia paneles específicos:
 - Rol VECINO: Accede a su perfil para: Consultar deuda y puntos, leer avisos del muro y chatear con el cobrador.
 - Rol COBRADOR: Entra a los módulos operativos para: Registrar cobro de agua (con sub-pasos para elegir pago y aplicar descuento), sugerir cobros usando IA, registrar asistencia a jornadas, añadir gastos, ver mapas de pagos por calles y usar un chat privado.

- Rol ADMINISTRADOR: Accede al panel administrativo para: Aprobar o rechazar solicitudes de acceso, gestionar cuentas de vecinos, configurar las tarifas/puntos y consultar logs de auditoría.
- Rol MINSA: Ingresa al módulo de reportes para: Ver balances financieros y descargar la información en formatos PDF o Excel.

3. Flujo de Salida:
- Desde cualquiera de los módulos finales de los distintos roles, se puede detonar la acción ¿Desea salir?.
 - Si la respuesta es Sí, se ejecuta la acción Cerrar Sesión y redirige al usuario a la Pantalla de Inicio de Sesión.
 - Si la respuesta es No, el usuario es regresado al Menú Principal.

21.3.2. Diagramas de Colaboración



Este diagrama ilustra específicamente el caso de uso: "Proceso de Cobro Offline-First". Nos muestra cómo interactúan las piezas de código cuando un cobrador registra un pago sin conexión a internet.

El Actor (El punto de partida):

- Cobrador (Offline): El usuario humano que interactúa con la aplicación.
- Mensaje [1]: "Ingresar data cobro". El actor crea la solicitud ingresando los datos en la interfaz. Objeto Boundary

(La Interfaz):

- CobrosPage.jsx: Recibe la acción del cobrador. Hace un auto-cálculo de la deuda en pantalla.
- Interacción: Envía el objeto con los datos del pago (pagoDTO) hacia la capa de negocio mediante un "Envío de Mutación" (registrarPago(pagoDTO)).

Objeto Control/Engine (El Cerebro de la operación):

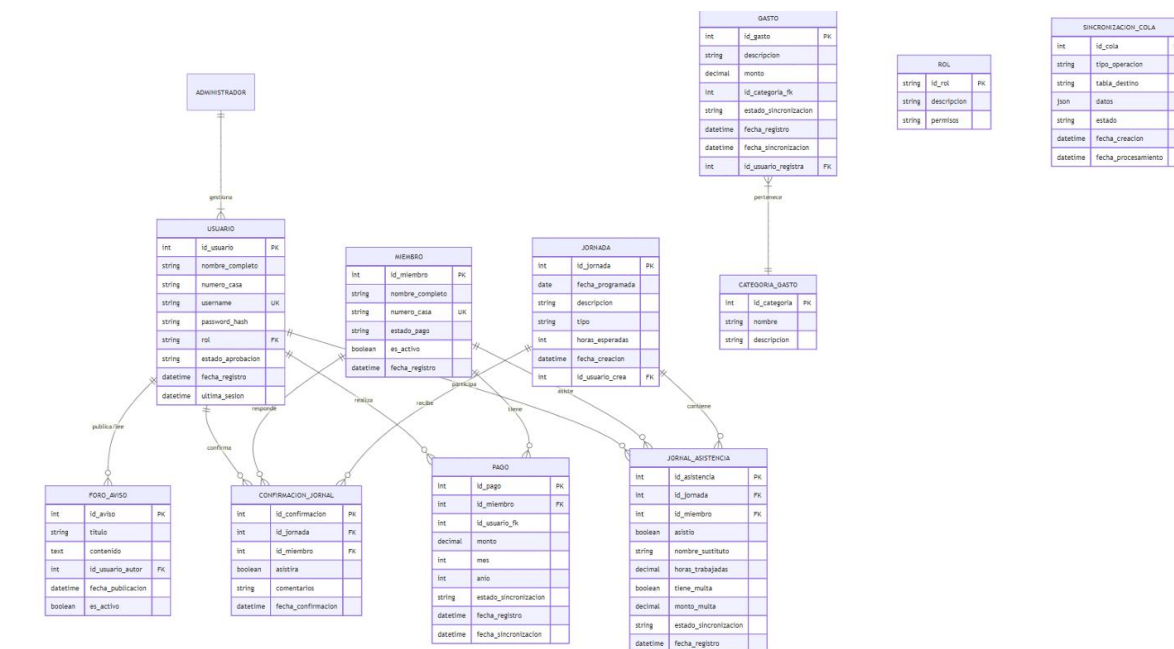
- pagosService.js: Es el motor principal que orquesta todo el proceso de cobrar. Recibe la petición desde la interfaz (Mensajes 2 y 3.1) y se comunica con varios otros servicios de forma colaborativa:
 - Consulta a aiService.js para hacer un análisis predictivo de morosidad.
 - Consulta a comisionesService.js para calcular la distribución del dinero (Motor de reglas declarativo, ej. Split 40/60).
 - Consulta a puntosService.js para gestionar la fidelización (sumar 100 puntos comunitarios).
 - Envía a baseIDB:db.js (DexieInstance) la orden de guardar la transacción localmente para persistencia en el dispositivo.
 - Envía la tarea pendiente al SyncQueueWorker (Web Worker) para que, de forma asíncrona, intente subir los datos a la nube cuando haya internet.

Retorno al usuario:

- Después de orquestar todas estas interacciones locales, pagosService.js envía el Mensaje [5]: "Notificar Cobro Exitoso" (Confirmación de Éxito) de vuelta a CobrosPage.jsx, para que el Cobrador vea en pantalla que su operación finalizó correctamente.

22. Modelo de base de datos

22.1. Modelo Conceptual

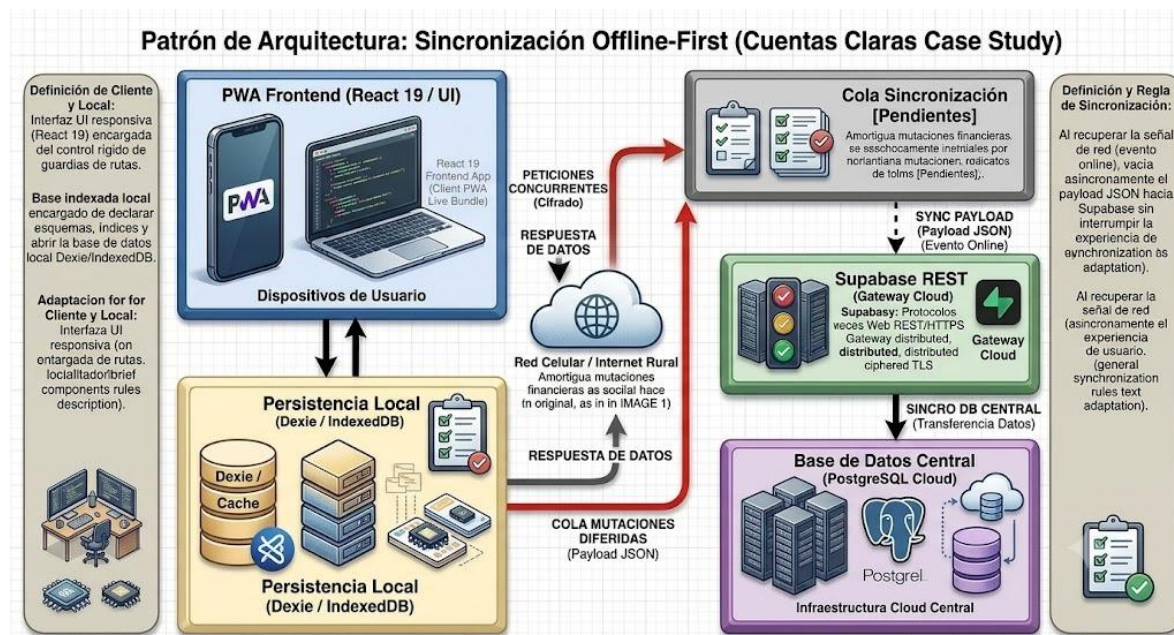


Entidades Core: USUARIO, MIEMBRO, PAGO, JORNADA, JORNAL_ASISTENCIA, GASTO, CATEGORIA_GASTO, SINCRONIZACION_COLA.

22.2. Modelo Lógico y Físico

- **PAGO**: `id_pago` (PK), `id_miembro` (FK), `monto` (decimal), `mes` (int), `anio` (int), `estado_sincronizacion` (string).
- **JORNAL_ASISTENCIA**: `id_asistencia`, `id_jornada`, `asistio` (boolean), `nombre_sustituto`, `monto_multa`.
- **SINCRONIZACION_COLA**: `idCola`, `tipo_operacion`, `tabla_destino`, `json datos`, `estado`. Los triggers de PostgreSQL (Supabase) actualizan las vistas materializadas como `vista_estado_miembros`.

22.3. Modelo Físico



23. Correspondencia de necesidades de actores y Casos de Uso

23.1. Matriz de trazabilidad

- Cobrador en el barro: Necesita no bloquear la app guardando datos -> Uso de IndexedDB con guardado optimista.
- Dirigente JAAR: Necesita detectar por qué bajó la recaudación -> CU-22 (Anomalías detectadas por IA).

24. Actores Y Sus Responsabilidades

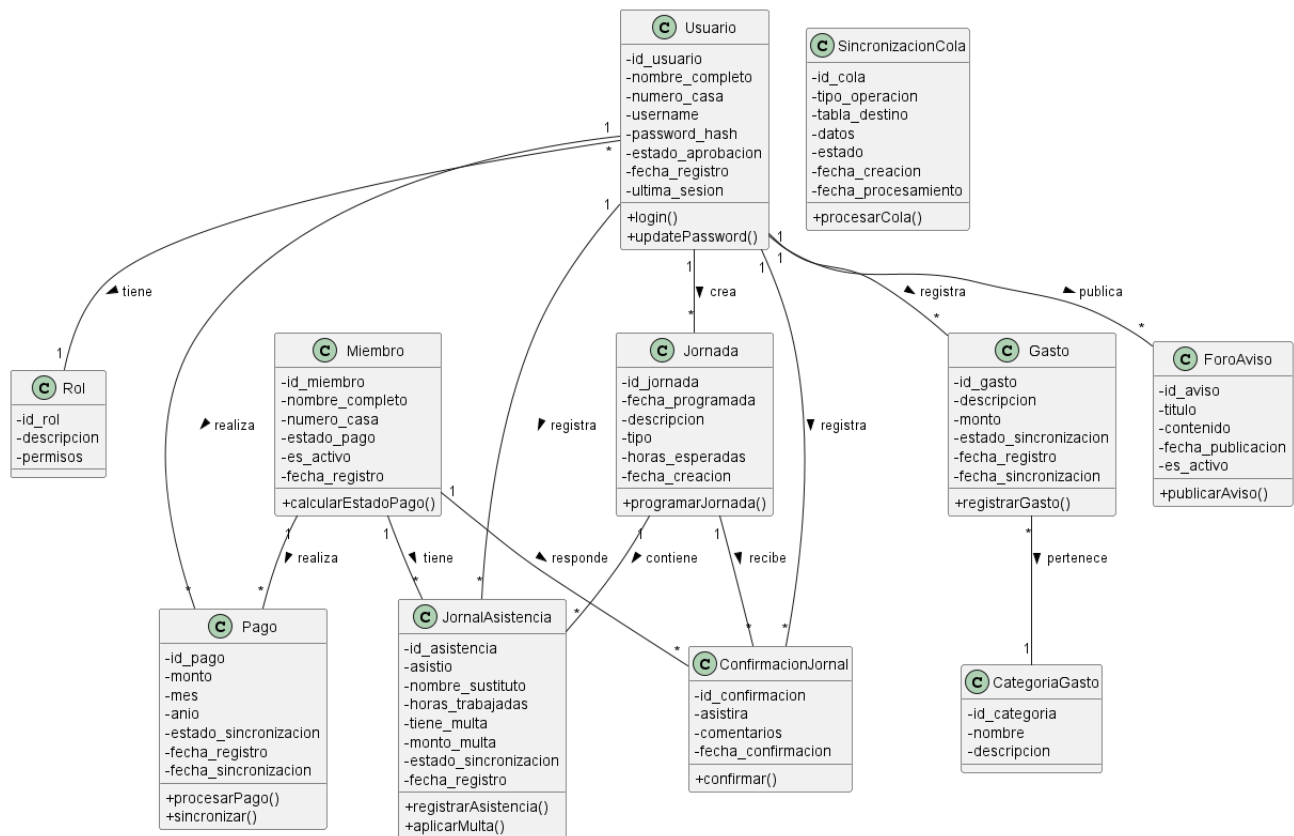
El Cobrador es responsable absoluto de la fidelidad del dinero en efectivo que lleva consigo y de realizar el Split de Comisión.

El Administrador responde legalmente ante el Auditor MINSA.

25. Modelo De Análisis

Los Context Providers en React (AuthContext) manejan el estado global del rol de usuario y el estado de la conexión a red del navegador (window.navigator.onLine), inyectando dependencias al Component Tree.

25.1. Diagrama de Clases (Modelo de Objetos)



El diagrama está compuesto por 11 clases principales que se relacionan entre sí para hacer funcionar el sistema. Sus especificaciones son:

- Módulo de Autenticación y Cuentas:

- o **Usuario**: Maneja las credenciales y el acceso al sistema. Contiene atributos como username, password_hash, estado de aprobación y última sesión. Tiene métodos para iniciar sesión (login()) y actualizar contraseñas.

- o **Rol**: Se relaciona uno-a-uno con el Usuario ("tiene"). Define los permisos y el nivel de acceso (por ejemplo, Administrador, Cobrador, Vecino).

- Módulo de Gestión de Vecinos y Pagos:

- o **Miembro**: Representa al vecino o propietario físico. Almacena su número de casa, si está activo y su estado general. Cuenta con el método calcularEstadoPago().

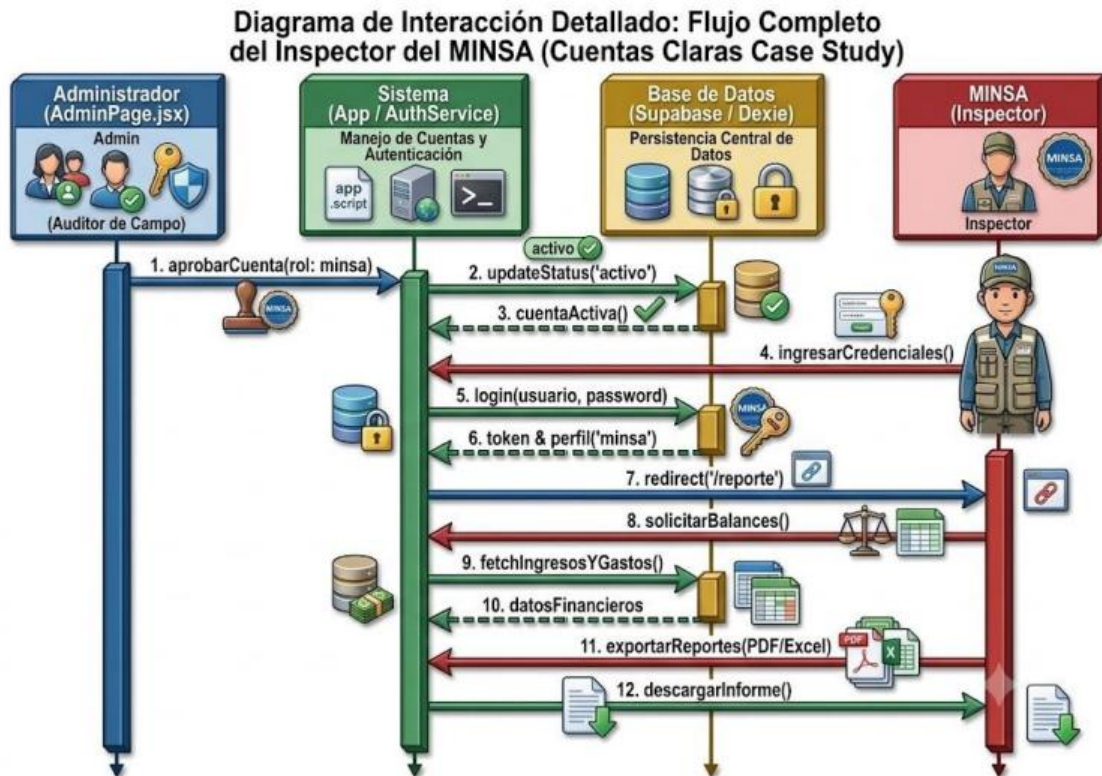
- o **Pago**: Registra las transacciones monetarias de los miembros. Guarda el monto, mes, año y su estado de sincronización. Está relacionado tanto con el Miembro que "realiza" el pago, como con el Usuario que lo registra.

- Módulo de Jornadas de Trabajo Comunitario:
 - o Jornada: Permite programar los días de trabajo comunitario, indicando fecha, descripción y horas esperadas.
 - o ConfirmacionJornal: Gestiona las respuestas de los miembros respecto a si asistirán o no a una jornada programada, incluyendo comentarios.
 - o JornalAsistencia: Registra la asistencia final el día de la jornada. Controla quién fue, si hubo sustitutos, las horas trabajadas, y si corresponde aplicar una multa y el monto de la misma.
- Módulo de Finanzas y Comunicación:
 - o Gasto: Registra los egresos operativos de la junta (monto y descripción). Un usuario los "registra".
 - o CategoriaGasto: Catálogo que clasifica o agrupa los distintos tipos de gastos del sistema.
 - o ForoAviso: Maneja las publicaciones del muro de noticias (título, contenido, fecha) que un Usuario "publica".
- Módulo Técnico / Soporte Offline:
 - o SincronizacionCola: Es una clase técnica aislada del modelo de negocio, diseñada para el soporte offline. Almacena temporalmente operaciones en cola (crear, actualizar, borrar), indicando la tabla destino y los datos en formato JSON, junto con su estado, para ser enviadas al servidor remoto cuando se recupere la conexión de internet a través de su método procesarCola()

25.2. Identificación de clases (entidad, frontera y control)

Descrito previamente.

25.3. Diagramas de Interacción



El diagrama cuenta con cuatro entidades principales (líneas de vida) que participan en el flujo:

- Administrador (AdminPage.jsx) / Auditor de Campo: Actor humano/interfaz que inicia el proceso de habilitar cuentas.
- Sistema (App / AuthService): El componente lógico que maneja las cuentas, la autenticación y la redirección.
- Base de Datos (Supabase / Dexie): El sistema de persistencia central donde se almacenan y consultan los datos.
- MINSA (Inspector): El usuario final en este caso de uso que interactúa con el sistema para obtener la información financiera.

o Flujo de interacción paso a paso:

- `aprobarCuenta(rol: minsa)`: El Administrador envía una solicitud al Sistema para aprobar una cuenta y asignarle el rol del MINSA.

- `updateStatus('activo')`: El Sistema se comunica con la Base de Datos para actualizar el estado de esa cuenta a "activo".
- `cuentaActiva()`: La Base de Datos confirma al Sistema (mediante un mensaje de retorno punteado) que la cuenta ha sido activada con éxito.
- `ingresarCredenciales()`: El Inspector del MINSA accede a la aplicación e ingresa sus credenciales hacia el Sistema.
- `login(usuario, password)`: El Sistema envía las credenciales proporcionadas a la Base de Datos para su validación.
- `token & perfil('minsa')`: Una vez validadas, la Base de Datos devuelve al Sistema el token de sesión y confirma el perfil de usuario correspondiente.
- `redirect('/reporte')`: El Sistema redirige la interfaz del Inspector del MINSA hacia la vista o página de reportes.
- `solicitarBalances()`: El Inspector del MINSA solicita ver los balances financieros a través del Sistema.
- `fetchIngresosYGastos()`: El Sistema ejecuta la consulta a la Base de Datos para obtener la información de ingresos y gastos.
- `datosFinancieros`: La Base de Datos retorna la información financiera solicitada al Sistema.
- `exportarReportes(PDF/Excel)`: El Inspector del MINSA solicita al Sistema la exportación de los datos visualizados en formatos PDF o Excel.
- `descargarInforme()`: El Sistema genera y entrega el documento, permitiendo la descarga del informe hacia el dispositivo del Inspector del MINSA.

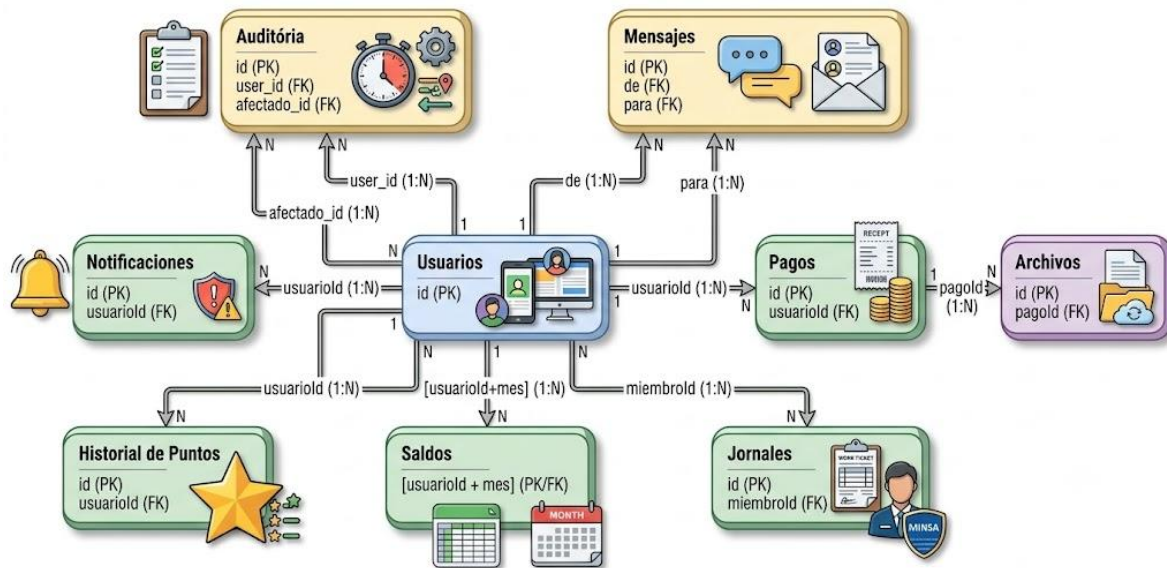
26. Análisis De Relaciones Entre Módulos

- El módulo Comisiones y Recompensas acopla con el Módulo de Pagos Flexibles a través de una transacción atómica (cuando se guarda el pago, se calcula el split y se guarda la comisión de inmediato).
- El módulo Ruta IA es lector (Read-Only) del padrón de MIEMBROS y PAGOS para calcular Z-scores, sin mutar datos.

26.1. Relación de Asociación

Diagrama de Asociaciones y Claves Foráneas (Cuentas Claras Case Study)

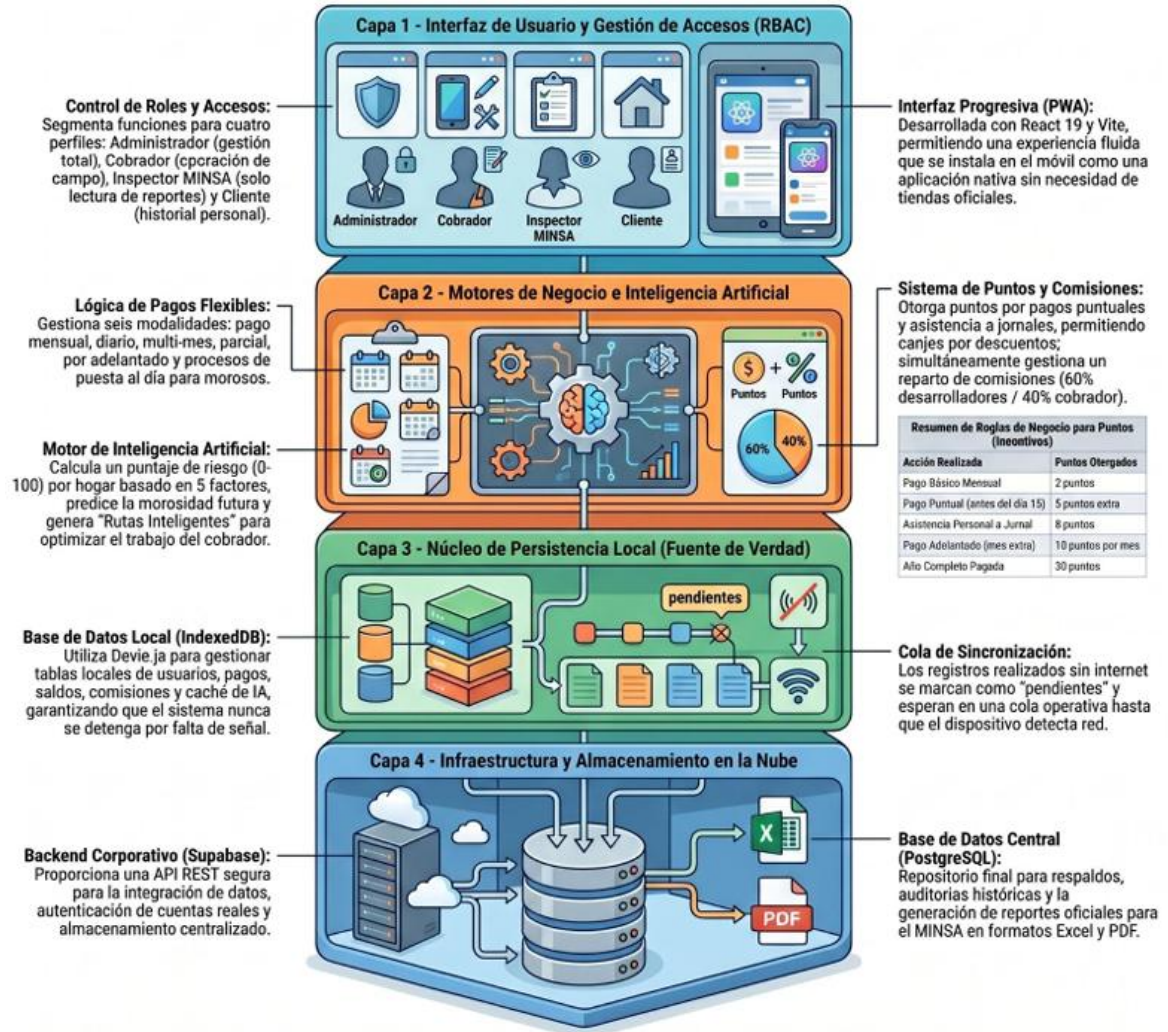
(Diagrama de Relaciones: Muestra las asociaciones, claves foráneas y la cardinalidad de los principales almacenes de datos, y los actores involucrados...)



27. Modelo OSSIM

- **Información:** Los datos críticos se almacenan de forma remota en PostgreSQL con respaldos diarios provistos por Supabase.
- **Mantenimiento:** El CI/CD configurado en el package.json corre Vite Build y Linters, permitiendo deploys con cero tiempos de inactividad (Zero-Downtime Deploy) al hacer Push a Main.

Arquitectura Integral JAAR Digital v2.0: Ecosistema "Cuentas Claras"

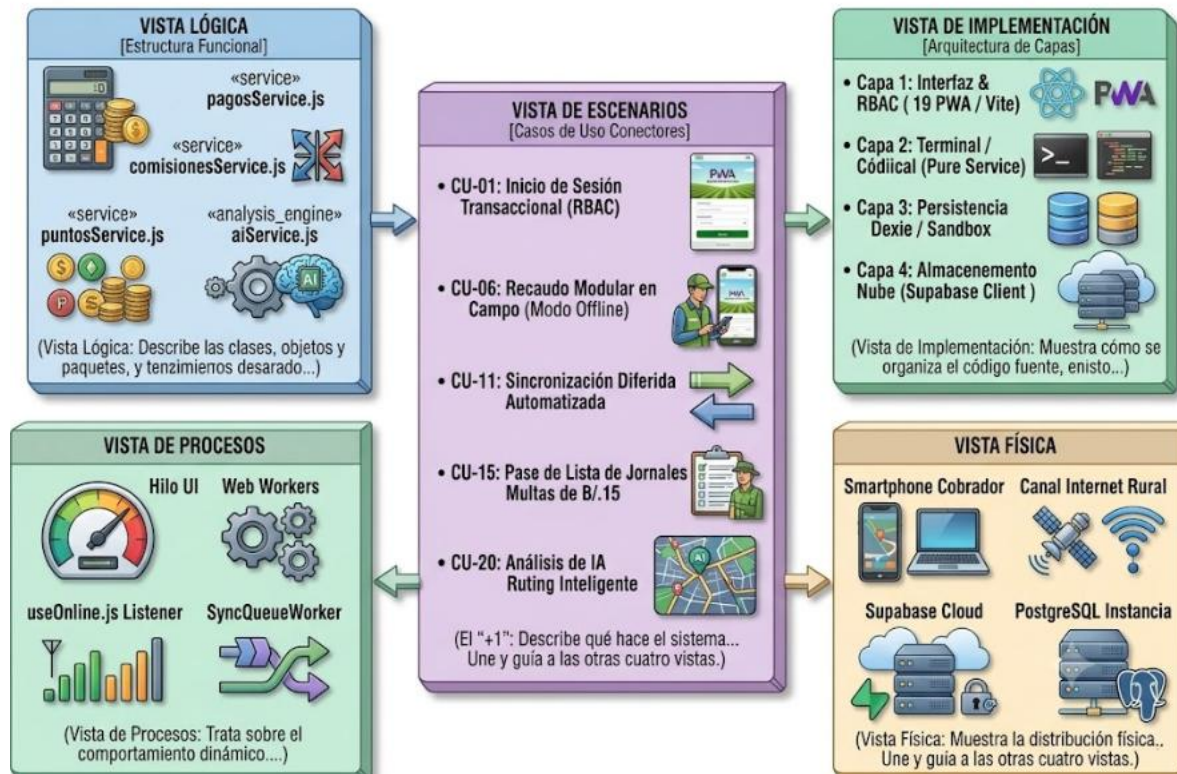


28. Documentación De La Arquitectura

Arquitectura Offline-First SPA. Los flujos de sincronización funcionan en Background, utilizando un Polling o un listener de evento de reconexión.

28.1. Modelo de “4+1” (diagrama de kruchten)

3.4.1 Modelo de “4+1 Vistas” (Cuentas Claras Case Study)



El diagrama representa la Capa de Lógica de Negocio (Servicios y Motores de Aplicación) del sistema, compuesta por módulos JavaScript que orquestan el flujo financiero, las comisiones y la fidelización. Se compone de cuatro servicios principales que interactúan entre sí:

- `pagosService.js` (<>): Actúa como el orquestador principal.
 - Atributos: Utiliza una instancia de base de datos local (- `baseIDB`: `DexieInstance`) y un objeto de configuración (- `m_config`: `Object`).
 - Métodos: Contiene las operaciones centrales como registrar pagos (+ `registrarPago`), calcular deudas (+ `calcularDeuda`), migrar datos heredados (+ `migrarDatosLegacy`) y obtener el historial de usuarios (+ `obtenerHistorial`).
- `comisionesService.js` (<>): Es invocado por el servicio de pagos para manejar la distribución de los fondos.
 - Atributos: Define internamente el split o división de tasas, con un 40% (- `splitTasa`: `Decimal = 0.40`) y un 60% (- `devTasa`: `Decimal = 0.60`).
 - Métodos: Se encarga de calcular dicha división (+ `calcularSplit4060`) y registrar la comisión correspondiente (+ `registrarComision`).

- puntosService.js (<>): Es invocado por el servicio de pagos para gestionar el sistema de fidelización comunitaria.
 - Atributos: Define la tasa de canje (- tasaCanje: Decimal = 0.10) y el límite máximo mensual (- limiteMaxMes: Decimal = 1.50).
 - Métodos: Permite otorgar puntos por pagos (+ otorgarPuntos), canjear puntos de los usuarios (+ canjearPuntos) y verificar bonos anuales (+ verificarBonosAnuales).
- aiService.js (<>): Es un motor inteligente analítico independiente que provee servicios de Análisis y Predicción al servicio de pagos.
 - Atributos: Posee un tiempo de vida de caché (- cacheTTL: int = 3600) y un arreglo de factores de posibilidad (- factoresPoss: Array).
 - Métodos: Contiene algoritmos complejos para calcular puntajes de riesgo (+ calcularPuntajeRiesgo), predecir morosidad de usuarios (+ predecirMorosidad), detectar anomalías estadísticas (+ detectarAnomaliasZScore) y generar colas de cobranza inteligentes (+ generarColaCobranza).

29. Vistas

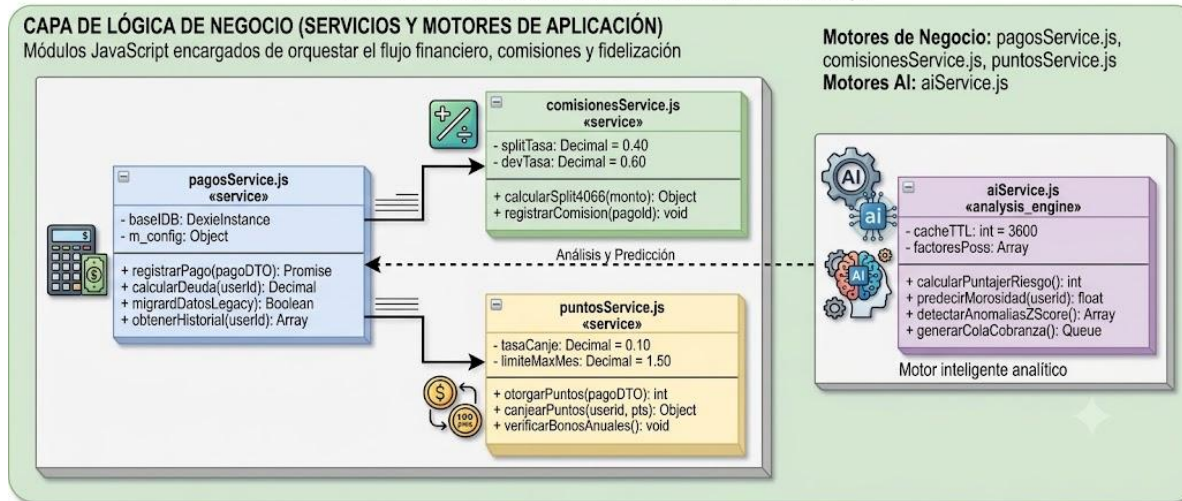
29.1. Vista Lógica

Dividido en: - utils/ (Helpers criptográficos, formateros). - services/ (Data Access Layer - Dexie/Supabase). - components/ (UI presentacional, botones y Toasts). - pages/ (Vistas por rol).

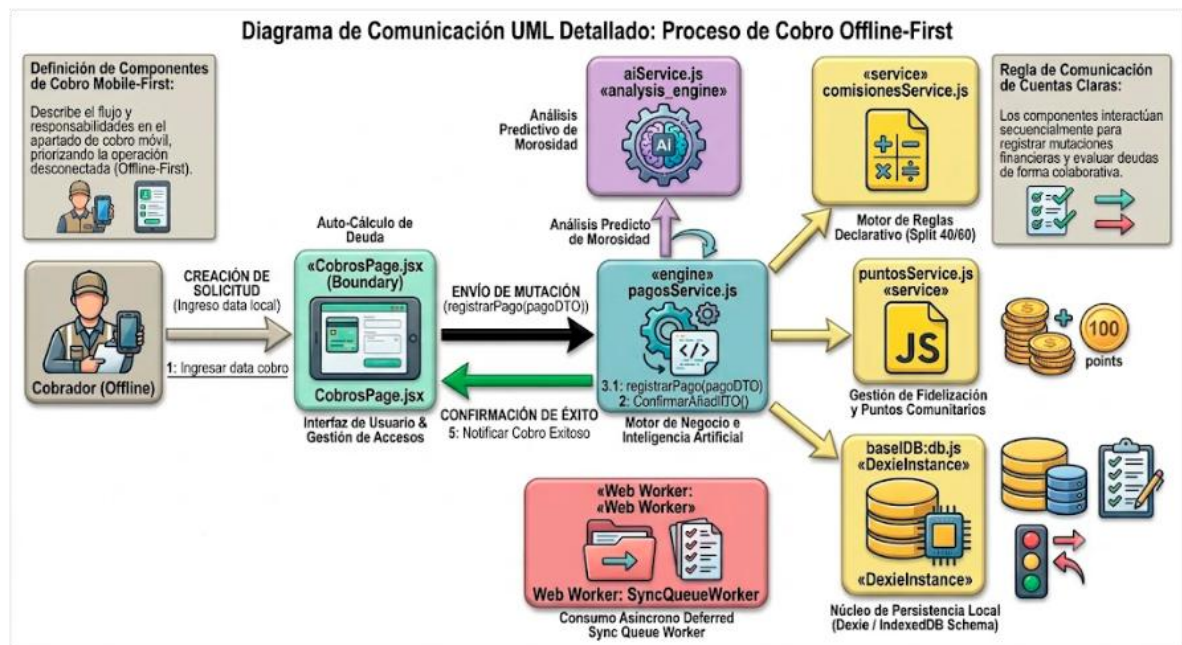
29.1.1. Diagrama de clases

3.4.3 Vista Lógica

Muestra los bloques de construcción funcionales del sistema en código.



29.1.2. Diagrama de comunicación



El diagrama ilustra el escenario de interacción durante una operación principal del sistema, como lo es la consulta de un vecino y el cálculo/registro de su estado de cuenta. El flujo de comunicación se desarrolla entre los siguientes objetos clave:

- El Actor (Cobrador/Administrador): Inicia el flujo interactuando con la interfaz gráfica.
- UI: React Components (Frontera/Vista): Recibe la acción del usuario (por ejemplo, buscar un usuario o registrar un pago). Envía el mensaje de solicitud hacia la capa de negocio.

- Servicio: pagosService (Controlador/Lógica): Actúa como el nodo central de comunicación. Al recibir la petición de la UI, orquesta una serie de mensajes:
 - Envía mensajes de consulta a la base de datos local (ej. obtenerHistorial()).
 - Comunica mensajes paralelos hacia comisionesService y puntosService para actualizar los beneficios o divisiones (splits) de los pagos.
 - Se comunica con aiService para solicitar el cálculo del riesgo (calcularPuntajeRiesgo()).
- BD Local: Dexie / IndexedDB (Persistencia): Recibe las consultas (Queries) del pagosService y devuelve los datos solicitados de forma inmediata (ya que es la fuente de verdad local).
- Cola de Sincronización y Supabase API (Nube): De forma asíncrona (como un subproceso), el servicio de persistencia emite un mensaje condicional hacia la API central de Supabase para reflejar los cambios solo si hay conexión a internet (pushSync()).

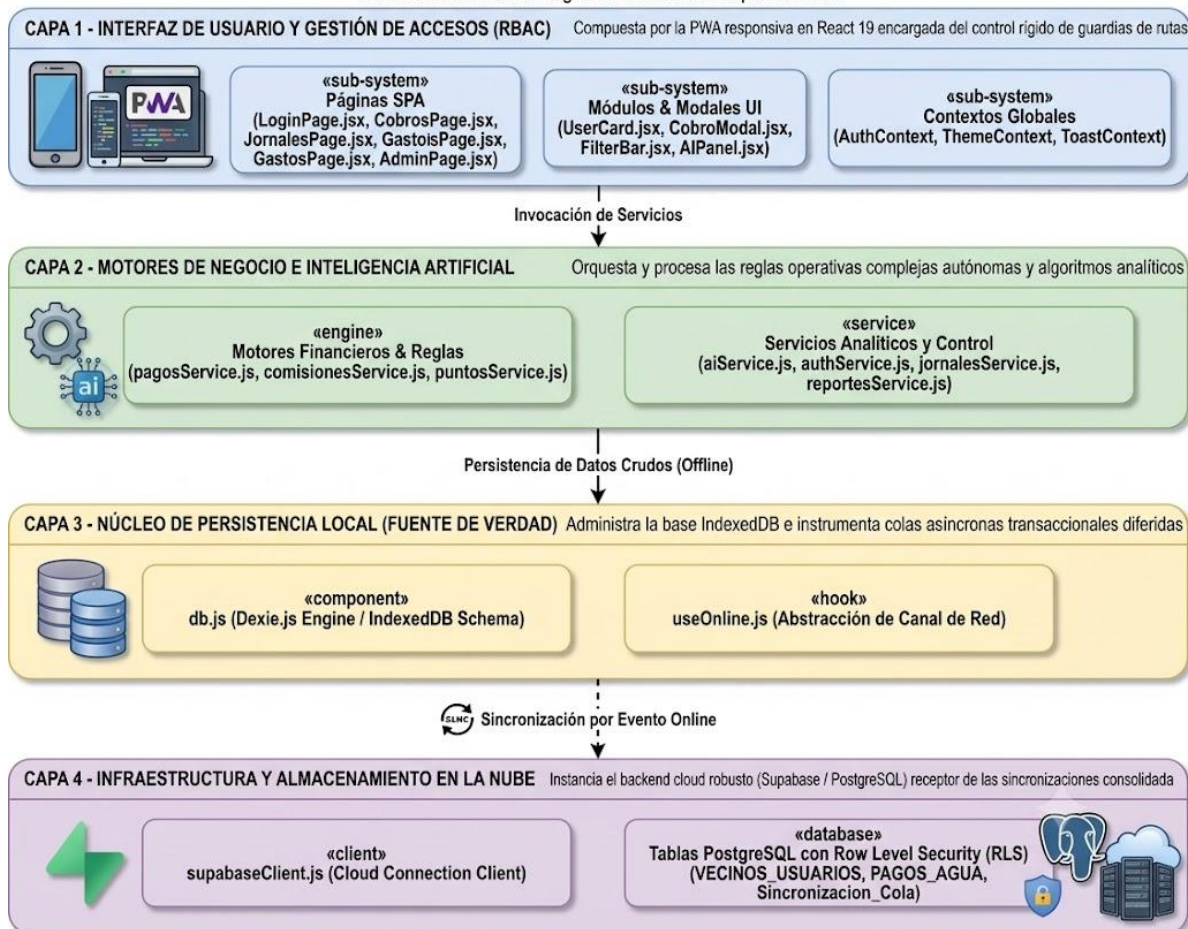
29.2. Vista de desarrollo

Empaquetado moderno ESM vía Vite. Dependencias clave: dexie-react-hooks, jspdf, @supabase/supabase-js, xlsx.

29.2.1. Diagrama de componentes

3.4.3.3 Vista de Implementación o Componentes (capas)

Describe la distribución lógica del software en capas elásticas.



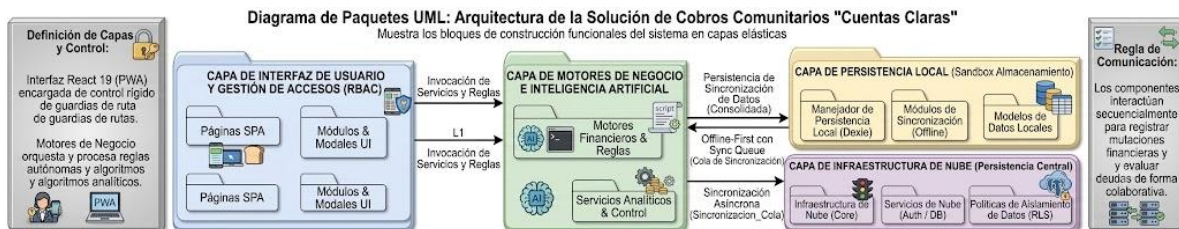
El diagrama de componentes representa la arquitectura interna del lado del cliente (Frontend y Motores lógicos locales). Los componentes modulares que interactúan colaborativamente en la aplicación son los siguientes:

- Componente de Presentación (UI Components / AIPanel.jsx): Es el bloque visual encargado de renderizar la interfaz SPA en React 19. Consume de forma directa las interfaces expuestas por los servicios de negocio para desplegar balances, métricas del recaudo y los indicadores gráficos de predicción de riesgo.
- Componente de Control de Acceso (authService.js): Encapsula el comportamiento del sistema de seguridad basado en roles (RBAC). Provee control estricto sobre las guardias de las rutas del sistema para restringir o permitir el acceso a las pantallas según el rol del usuario autenticado.
- Módulo Central de Operaciones (pagosService.js): Constituye el componente neurálgico para el flujo monetario. Expone métodos críticos para registrar abonos mensuales o parciales, calcular deudas del hogar y

estructurar los estados de cuentas colectivas. Mantiene dependencias de uso con los componentes accesorios de cálculo.

- Componentes de Apoyo de Negocio (comisionesService.js y puntosService.js): Son componentes de lógica acoplados al flujo de cobros. comisionesService.js automatiza el cálculo matemático del split porcentual entre el cobrador y los desarrolladores por cada transacción. Por su parte, puntosService.js asigna puntajes por la asistencia a jornadas comunitarias (jornales) o por la puntualidad del pago, controlando las reglas fijas de canjes y deducciones.
- Motor Analítico Inteligente (aiService.js): Componente avanzado que procesa la telemetría estadística de los usuarios. Evalúa los factores predictivos de morosidad mediante fórmulas complejas, detecta anomalías transaccionales utilizando la técnica estadística de puntuación estándar (Z-Score) y organiza dinámicamente la cola estructurada de cobranzas efectivas.
- Componente de Persistencia Local (db.js / Dexie): Es el cimiento de la arquitectura "Offline-First". Los módulos lógicos financieros y de analítica interactúan con este componente, que encapsula el acceso a la base de datos IndexedDB en el navegador del dispositivo, asegurando el almacenamiento local continuo y gestionando las colas asíncronas para el push definitivo hacia la nube cuando el dispositivo recupera acceso a internet.

29.2.2. Diagrama de paquetes



El diagrama de Paquetes UML de la solución "Cuentas Claras" divide el sistema en cuatro paquetes o capas principales:

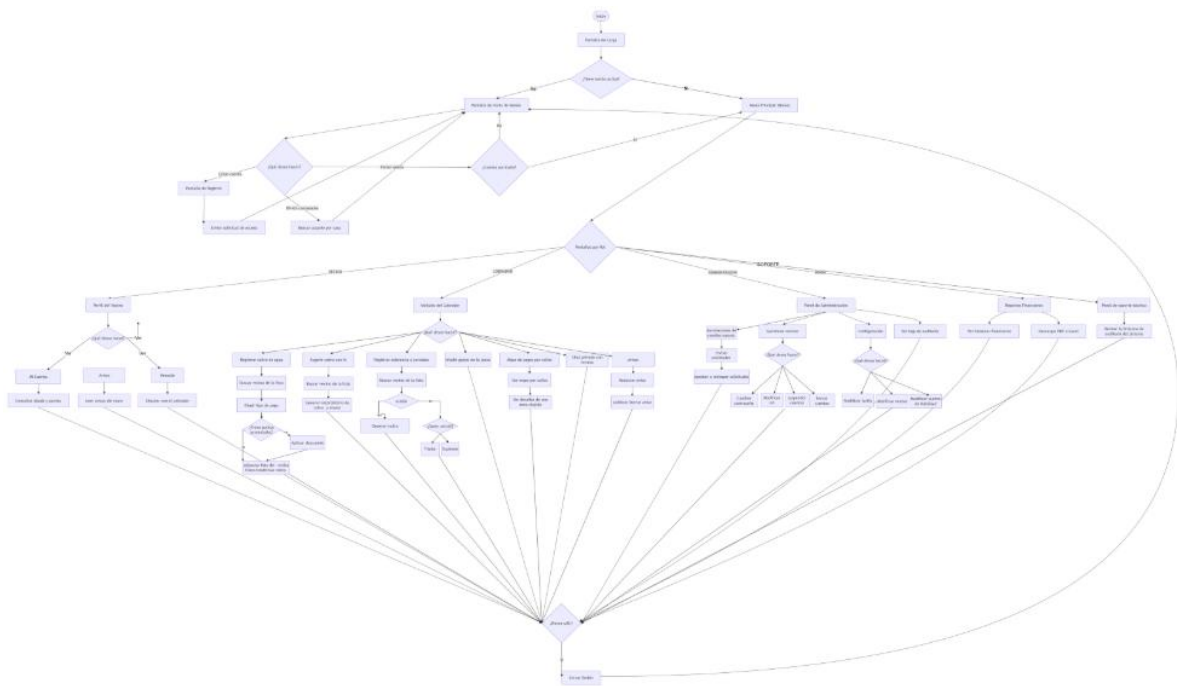
- Capa de Interfaz de Usuario y Gestión de Accesos (RBAC): Es la interfaz construida en React 19 (PWA) encargada del control rígido de las guardias de rutas. Internamente agrupa los paquetes de "Páginas SPA" y "Módulos & Modales UI". Esta capa se encarga de realizar la "Invocación de Servicios y Reglas" (L1) hacia la capa de negocio.
- Capa de Motores de Negocio e Inteligencia Artificial: Es el núcleo que orquesta y procesa las reglas autónomas y los algoritmos analíticos. Contiene los subpaquetes de "Motores Financieros & Reglas" y "Servicios Analíticos & Control". Actúa como intermediario gestionando el flujo de datos hacia las capas de persistencia.

- Capa de Persistencia Local (Sandbox Almacenamiento): Es el pilar de la estrategia "Offline-First" y funciona con una "Sync Queue" (Cola de Sincronización). Incluye el "Manejador de Persistencia Local (Dexie)", los "Módulos de Sincronización (Offline)" y los "Modelos de Datos Locales". Recibe y envía datos a la capa de negocio para mantener una persistencia consolidada.
- Capa de Infraestructura de Nube (Persistencia Central): Actúa como el repositorio definitivo y seguro. Agrupa la "Infraestructura de Nube (Core)", los "Servicios de Nube (Auth / DB)" y aplica "Políticas de Aislamiento de Datos (RLS)". Recibe la "Sincronización Asíncrona (Sincronizacion_Cola)" directamente desde la capa de motores de negocio.

29.3. Vista de procesos

Flujo de Sincronización (Diagrama): 1. User acciona Guardar. 2. Si NO red -> IndexedDB -> SINCRONIZACION_COLA estado=Pendiente. 3. Al recuperar Red -> Evento despachado -> Worker lee cola -> POST Supabase. 4. Éxito -> Limpia Cola.

29.3.1. Diagrama de actividad o secuencia



<https://drive.google.com/file/d/1mOHkmQztAWtCNLthnkBD6lu9jAXqHI8j/view?usp=sharing>

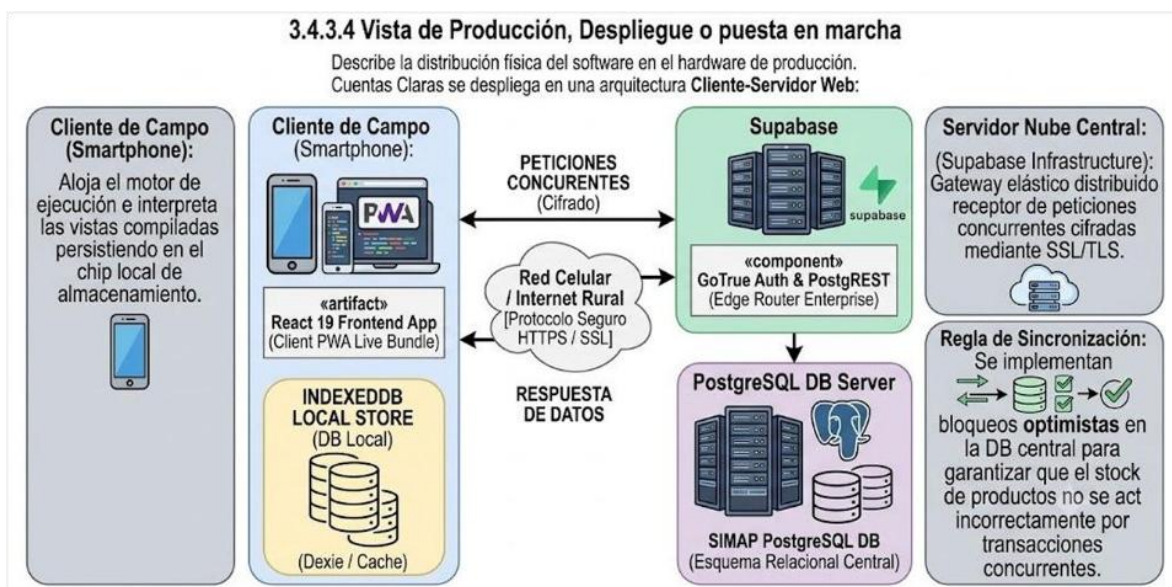
Desde mi rol como desarrollador, estos diagramas los diseño como un flujo secuencial. Defino un punto de inicio (nodos iniciales), acciones concretas de

procesamiento de datos, puntos de decisión (nodos de decisión) que condicionan el camino, y un nodo final de cierre. Aquí no me preocupo por la interacción entre módulos externos, sino por la correcta ejecución de la lógica interna de esa función específica.

29.4. Vista física o de despliegue

Hosting de Assets estáticos de React en Vercel (CDN global). Base de datos en Supabase (AWS backend PostgreSQL). Dispositivos de usuario corren el runtime de JavaScript localmente.

29.4.1. Diagrama de despliegue



El diagrama de despliegue del sistema SIMAP Digital está estructurado en dos nodos principales que se comunican de forma asíncrona debido a su naturaleza Offline-First:

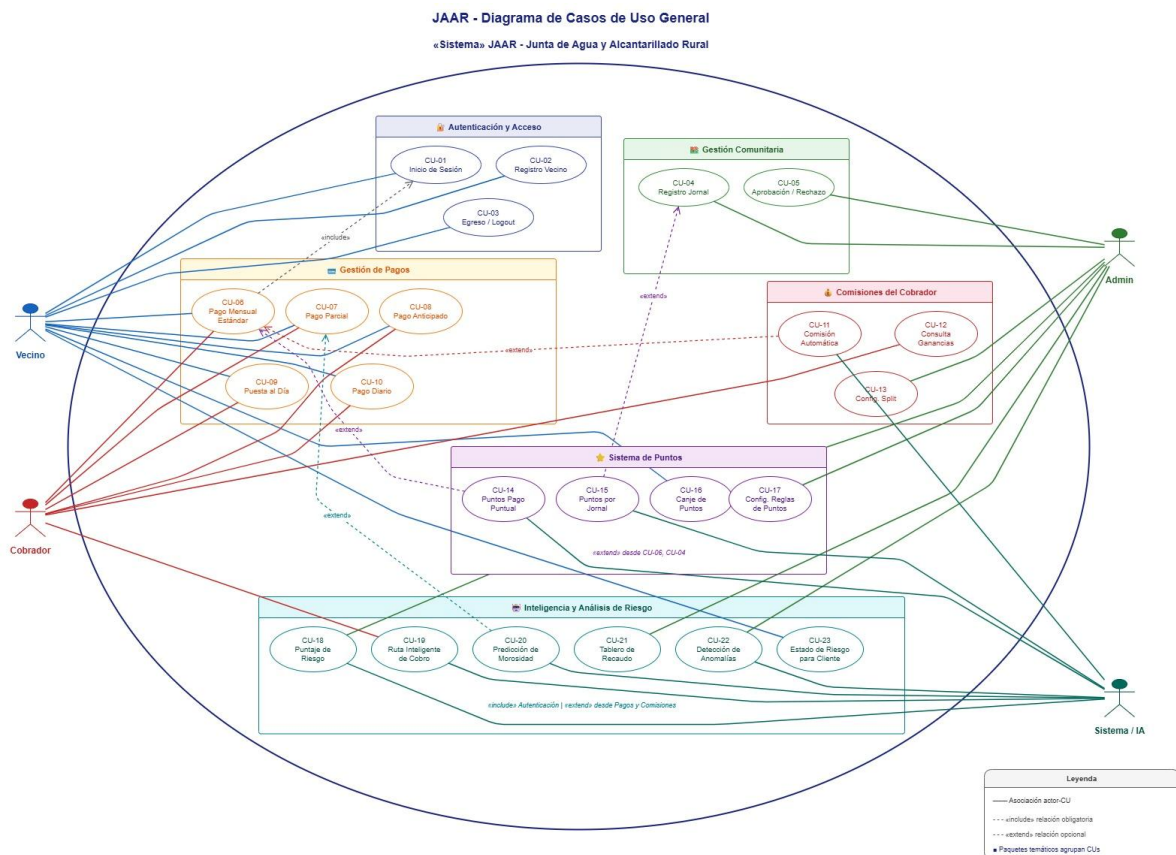
- **Nodo Cliente (Dispositivo del Usuario / Terminal Web):** Este nodo representa el smartphone o computadora del cobrador, administrador o vecino. En él se ejecuta un entorno de Navegador Web que aloja la aplicación Frontend (PWA) construida con React 19 y Vite. Dentro de este entorno local, se encuentra un artefacto crucial: la base de datos local IndexedDB (gestionada por Dexie.js), la cual actúa como la fuente de verdad inmediata, permitiendo guardar pagos, jornales y usuarios sin necesidad de internet. También incluye el módulo local SheetJS para generar reportes en Excel directamente en el dispositivo.

- Nodo Nube (Servidor Supabase): Este nodo centralizado representa el backend del sistema. Contiene la API REST que recibe los datos, el servidor de Autenticación (Supabase Auth) para el control de roles reales, y el gestor de base de datos relacional PostgreSQL, que almacena de forma definitiva la información de toda la comunidad.
- Comunicación (Enlace de Red): Ambos nodos se conectan a través de Internet (vía protocolo HTTP/HTTPS). La comunicación es mediada por una "Cola de Sincronización"; el sistema cliente escucha los eventos de red y, al detectar conexión (online), hace un push de los registros marcados como pendientes en la base de datos local hacia la API de Supabase, confirmando su estado como sincronizado.

29.5. Escenarios

(Vista "+1"). - El hilo conductor. Validar que la arquitectura física e infraestructura soporta el Caso de Uso de "Alta demanda en Telemedicina matutina" (en caso de proyectos médicos) o en nuestro caso "Sincronización masiva de fin de jornada del Cobrador".

29.5.1. Diagramas de caso de uso general



Glosario

- PWA: Aplicación Web Progresiva.
- Offline-First: Patrón de diseño donde la aplicación está concebida primordialmente para funcionar sin red.
- Jornal (Faina): Obligación comunitaria de trabajo físico en el acueducto.
- Z-Score: Medida estadística utilizada por la Ruta IA para identificar caídas drásticas de recaudo.

Historial de Versiones

Fecha	Versión	Autor	Descripción
14/07/2026	2.1	Equipo SIMAP	Integración de etiquetas para inserción posterior de diagramas.

Información del Proyecto

Atributo	Valor
Proyecto	SIMAP Digital Cuentas Claras
Fecha	14/07/2026
Regulador	Ministerio de Salud (MINSa)

Aprobaciones

Nombre	Cargo	Organización	Firma
Directiva	Presidente	JAAR Panamá	_____